

Rhombix Technologies DevOps Internship:

Intern Name: Muhammad Jami Sultani.

Domain: DevOps.

Internship Program: Rhombix Technologies 2025.

Task Deadline: 31 October 2025.

Task 2:

Introduction to Continuous Integration (CI) with Jenkins.

Objective:

The purpose of this task is to understand the concept of **Continuous Integration (CI)** using **Jenkins**.

In this project, you will learn how to:

- Set up Jenkins locally or on a cloud instance (Ubuntu EC2 or local machine).
- Create a Jenkins pipeline connected with GitHub.
- Automate build and testing processes whenever code is pushed to GitHub.

This task demonstrates how CI helps teams detect issues early, improves code quality, and enhances collaboration between developers and operations.

◆ Step 1: Install Git.

Commands:

sudo apt update:

refreshes package lists.

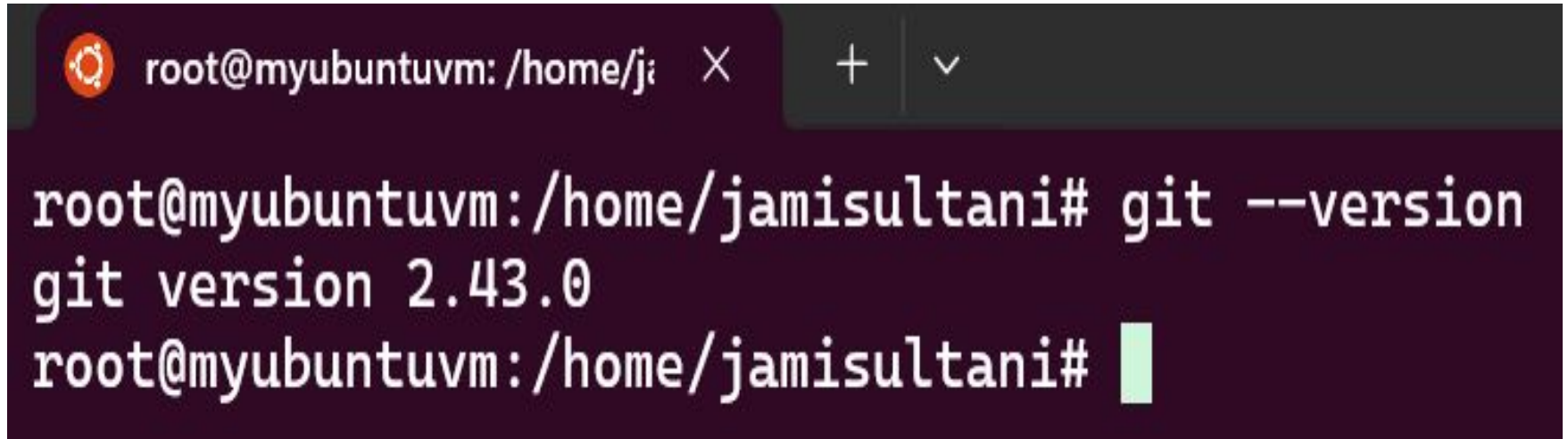
sudo apt install git -y:

installs Git non-interactively.

```
jamisultani@myubuntuvm: ~  
jamisultani@myubuntuvm:~$ sudo apt update  
[sudo] password for jamisultani:  
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease  
Hit:2 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease  
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease  
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,270 kB]  
Get:6 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [211 kB]  
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]  
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [8,984 B]  
Get:9 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2,080 kB]  
Get:10 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [470 kB]  
Get:11 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]  
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [905 kB]  
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [203 kB]  
Get:14 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]  
Get:15 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]  
Fetched 5,347 kB in 5s (1,125 kB/s)  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
4 packages can be upgraded. Run 'apt list --upgradable' to see them.  
jamisultani@myubuntuvm:~$
```

```
root@myubuntuvm: /home/jami# sudo apt install git -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
git is already the newest version (1:2.43.0-1ubuntu7.3).  
git set to manually installed.  
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.  
root@myubuntuvm: /home/jami#
```

git --version: confirms Git is installed and shows the version.

A terminal window with a dark purple background. The title bar at the top shows a terminal icon, the text 'root@myubuntuvvm: /home/ji', and window control buttons (close, maximize, and a dropdown arrow). The terminal content shows the command 'git --version' being executed, resulting in the output 'git version 2.43.0'. The prompt 'root@myubuntuvvm: /home/jamisultani#' is visible on both lines.

```
root@myubuntuvvm: /home/jamisultani# git --version
git version 2.43.0
root@myubuntuvvm: /home/jamisultani#
```

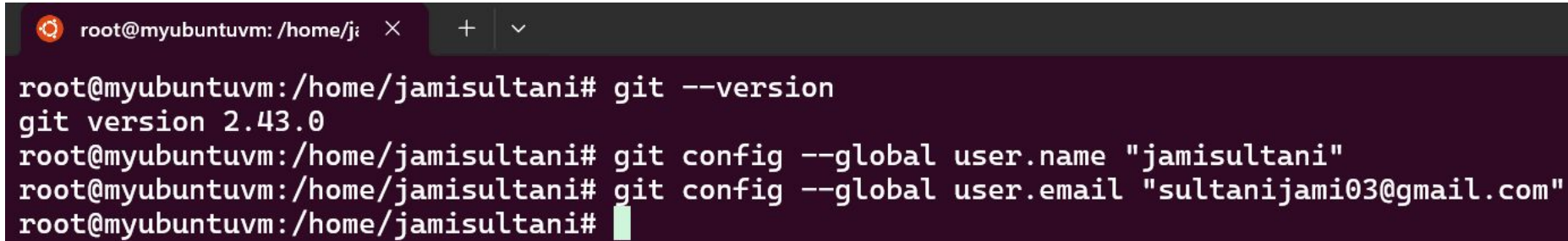
◆ Step 2: Configure Git (set global username & email).

Commands:

```
git config --global user.name  
"jamisultani"
```

```
git config --global user.email  
"sultanijami03@gmail.com"
```

.These set your global Git identity (name and email) so commits are tagged with this info. **--global** makes these settings apply to all repositories for this user.



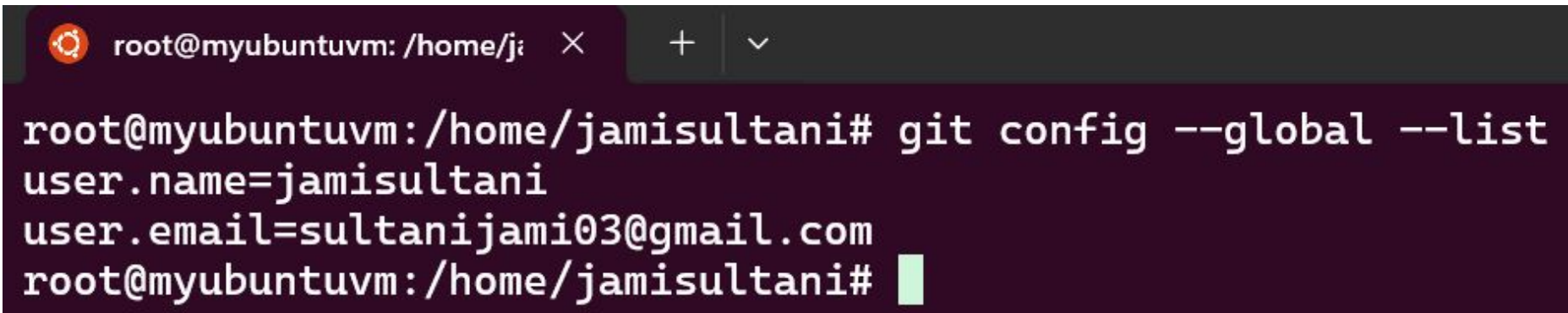
```
root@myubuntuvvm: /home/ji  ×  +  ∨  
root@myubuntuvvm:/home/jamisultani# git --version  
git version 2.43.0  
root@myubuntuvvm:/home/jamisultani# git config --global user.name "jamisultani"  
root@myubuntuvvm:/home/jamisultani# git config --global user.email "sultanijami03@gmail.com"  
root@myubuntuvvm:/home/jamisultani#
```

◆ Step 3: Verify Git configuration.

Commands:

git config --global --list:

Shows all globally configured Git settings (including the user.name and user.email you just set).

A terminal window with a dark purple background. The title bar shows 'root@myubuntuvm: /home/ji' with a close button. The terminal content shows the command 'git config --global --list' being executed, resulting in two lines of output: 'user.name=jamisultani' and 'user.email=sultanijami03@gmail.com'. The prompt 'root@myubuntuvm: /home/jamisultani#' is visible at the end of the second line, followed by a green cursor bar.

```
root@myubuntuvm: /home/jamisultani# git config --global --list
user.name=jamisultani
user.email=sultanijami03@gmail.com
root@myubuntuvm: /home/jamisultani#
```

◆ Step 4: Install Java JDK LTS Version - OpenJDK 21.

Commands:

sudo apt update:

refreshes the Ubuntu repositories for the latest software versions.

**sudo apt install
openjdk-21-jdk -y:**

installs the latest Long-Term Support (LTS) version of Java (JDK 21).

```
root@myubuntuv: /home/jami ✕ + ▼

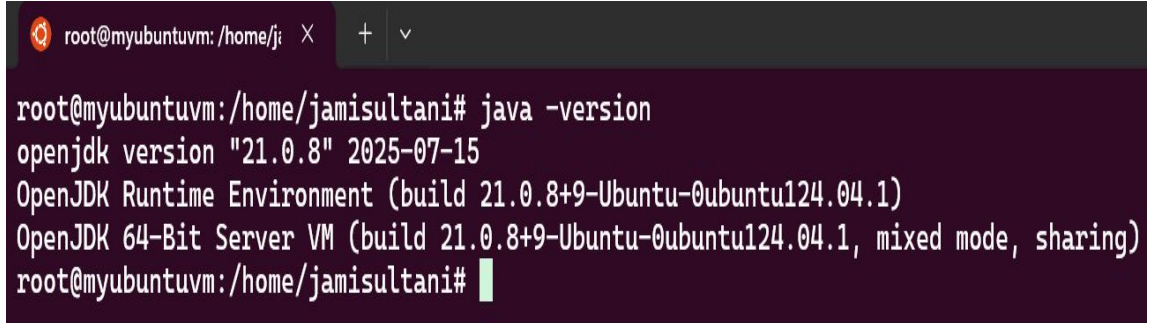
root@myubuntuv: /home/jamisultani# sudo apt update
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@myubuntuv: /home/jamisultani#
```

```
root@myubuntuv: /home/jami ✕ + ▼

root@myubuntuv: /home/jamisultani# sudo apt install openjdk-21-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates-java fonts-dejavu-extra java-common libatk-wrapper-java libatk-wrapper-java-jni libgif7 libice-dev
  libpcsc-lite libpthread-stubs0-dev libsm-dev libx11-dev libxau-dev libxcb1-dev libxdmcp-dev libxext-dev
  libxft-dev libxkbfile-dev libxml2-dev libxslt-dev libxss-dev libxt-dev libxtst-dev libz-dev libz-dev
  openjdk-21-jdk-headless openjdk-21-jre openjdk-21-jre-headless x11proto-dev xorg-sgml-doctools xtrans-dev
Suggested packages:
  default-jre libice-doc pcscd libsm-doc libx11-doc libxcb-doc libxtd-doc openjdk-21-demo openjdk-21-source visualvm
  libnss-mdns fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei | fonts-wqy-zenhei fonts-indic
The following NEW packages will be installed:
  ca-certificates-java fonts-dejavu-extra java-common libatk-wrapper-java libatk-wrapper-java-jni libgif7 libice-dev
  libpcsc-lite libpthread-stubs0-dev libsm-dev libx11-dev libxau-dev libxcb1-dev libxdmcp-dev libxext-dev openjdk-21-jdk
  openjdk-21-jdk-headless openjdk-21-jre openjdk-21-jre-headless x11proto-dev xorg-sgml-doctools xtrans-dev
0 upgraded, 22 newly installed, 0 to remove and 4 not upgraded.
Need to get 135 MB of archives.
After this operation, 316 MB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 ca-certificates-java all 20240118 [11.6 kB]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 fonts-dejavu-extra all 2.37-8 [1,947 kB]
Get:3 http://security.ubuntu.com/ubuntu noble-security/main amd64 openjdk-21-jre-headless amd64 21.0.8+9-us1-0ubuntu1-24.04.1 [46.4 MB]
Get:4 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 java-common all 0.75+expi [6,798 B]
Get:5 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libatk-wrapper-java all 0.40.0-3build2 [54.3 kB]
Get:6 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libatk-wrapper-java-jni amd64 0.40.0-3build2 [46.4 kB]
Get:7 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libgif7 amd64 5.2.2-1ubuntu1 [35.2 kB]
Get:8 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 xorg-sgml-doctools all 1:1.11-1.1 [10.9 kB]
Get:9 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 x11proto-dev all 2023.2-1 [602 kB]
Get:10 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libice-dev amd64 2:1.0.10-1build3 [51.0 kB]
Get:11 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libpcsc-lite amd64 2.0.3-1build1 [21.4 kB]
Get:12 http://pk.archive.ubuntu.com/ubuntu noble/main amd64 libpthread-stubs0-dev amd64 0.4-1build3 [4,746 B]
```

java -version:

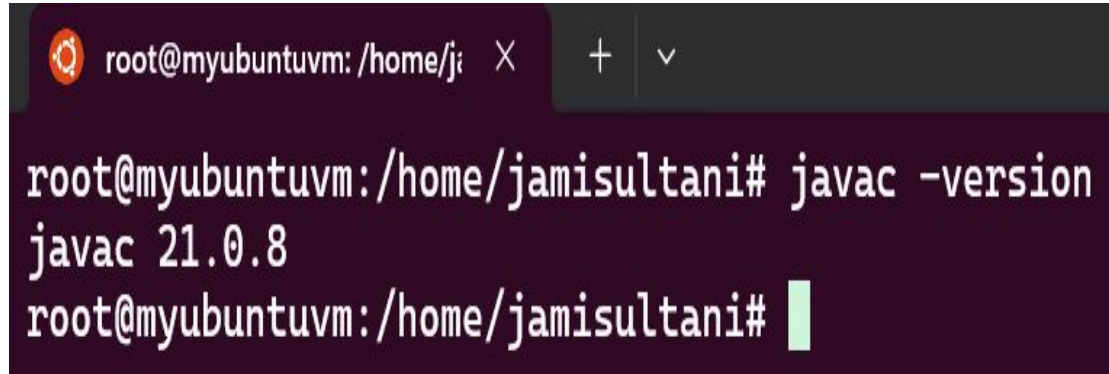
verifies that Java Runtime is installed.

A terminal window with a dark purple background. The title bar shows 'root@myubuntuvvm: /home/ji' and a close button. The terminal text shows the command 'java -version' and its output: 'openjdk version "21.0.8" 2025-07-15', 'OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)', and 'OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)'. The prompt returns to 'root@myubuntuvvm: /home/jamisultani#'.

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# java -version
openjdk version "21.0.8" 2025-07-15
OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)
OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)
root@myubuntuvvm:/home/jamisultani#
```

javac -version:

verifies that the Java compiler is installed and working.

A terminal window with a dark purple background. The title bar shows 'root@myubuntuvvm: /home/ji' and a close button. The terminal text shows the command 'javac -version' and its output: 'javac 21.0.8'. The prompt returns to 'root@myubuntuvvm: /home/jamisultani#'.

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# javac -version
javac 21.0.8
root@myubuntuvvm:/home/jamisultani#
```

◆ Step 5: Configure Java Globally (Set JAVA_HOME and PATH).

Commands:

sudo update-alternatives --config java:

lists installed Java versions and shows their paths.

```
root@myubuntuvvm: /home/ji  ×  +  ∨

root@myubuntuvvm:/home/jamisultani# sudo update-alternatives --config java
There is 1 choice for the alternative java (providing /usr/bin/java).

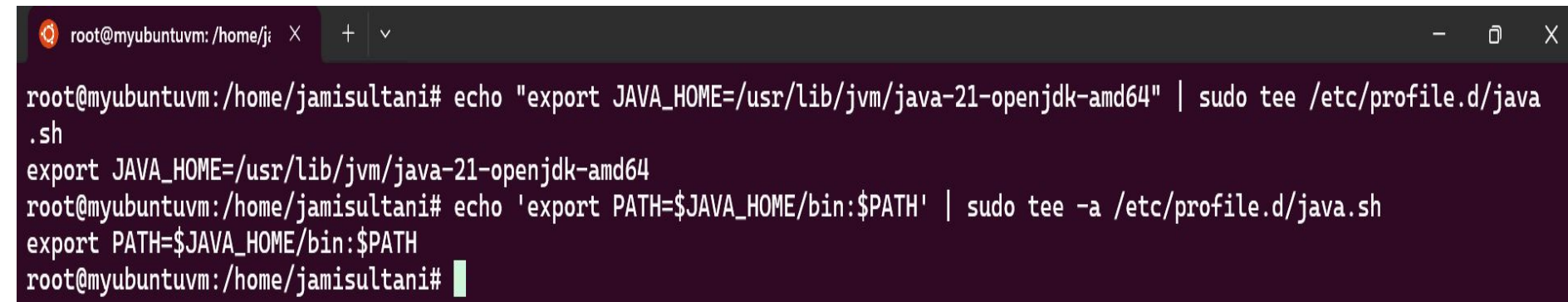
  Selection    Path
  -----
*  0           /usr/lib/jvm/java-21-openjdk-amd64/bin/java    2111    auto mode
   1           /usr/lib/jvm/java-21-openjdk-amd64/bin/java    2111    manual mode

Press <enter> to keep the current choice[*], or type selection number:
root@myubuntuvvm:/home/jamisultani#
```

```
echo "export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64"  
| sudo tee /etc/profile.d/java.sh:
```

```
echo 'export PATH=$JAVA_HOME/bin:$PATH' | sudo tee -a  
/etc/profile.d/java.sh:
```

stores global environment variables (**JAVA_HOME** and **PATH**) so Java can be used system-wide.



A terminal window with a dark background and light text. The window title bar shows 'root@myubuntuvvm: /home/ji' and standard window controls. The terminal output shows the following commands and their results:

```
root@myubuntuvvm:/home/jamisultani# echo "export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64" | sudo tee /etc/profile.d/java  
.sh  
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64  
root@myubuntuvvm:/home/jamisultani# echo 'export PATH=$JAVA_HOME/bin:$PATH' | sudo tee -a /etc/profile.d/java.sh  
export PATH=$JAVA_HOME/bin:$PATH  
root@myubuntuvvm:/home/jamisultani#
```

sudo chmod +x /etc/profile.d/[java.sh](#):

source /etc/profile.d/[java.sh](#)

loads the new environment without rebooting.



root@myubuntuvms: /home/jamisultani



```
root@myubuntuvms:/home/jamisultani# sudo chmod +x /etc/profile.d/java.sh
root@myubuntuvms:/home/jamisultani# source /etc/profile.d/java.sh
root@myubuntuvms:/home/jamisultani#
```

◆ Step 6: Verify Java Global Configuration.

Commands:

echo \$JAVA_HOME:

Shows the Java home directory path that you configured globally.

```
root@myubuntuvvm: /home/jamisultani# echo $JAVA_HOME
root@myubuntuvvm: /home/jamisultani#
```

which java:

Displays the full path of the Java binary file being used by the system.

```
root@myubuntuvvm: /home/jamisultani# which java
/usr/bin/java
root@myubuntuvvm: /home/jamisultani#
```

java -version:

Confirms the installed Java version and verifies it's working correctly.

```
root@myubuntuvvm: /home/jamisultani# java -version
openjdk version "21.0.8" 2025-07-15
OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)
OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)
root@myubuntuvvm: /home/jamisultani#
```

◆ Step 7: Install Apache Maven (Latest Version).

Commands:

sudo apt update:

refreshes package repositories
to ensure the latest Maven
version is available.

sudo apt install maven -y:

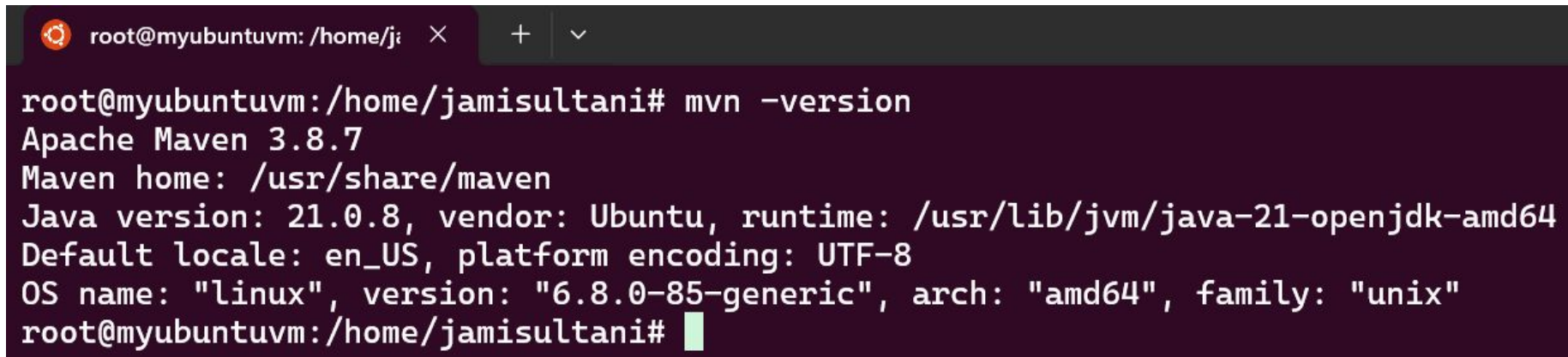
installs Apache Maven without
asking for confirmation.

```
root@myubuntuvm: /home/jamisultani# sudo apt update
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Fetched 200 kB in 2s (103 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@myubuntuvm: /home/jamisultani#
```

```
root@myubuntuvm: /home/jamisultani# sudo apt install maven -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libaopalliance-java libapache-pom-java libatinject-jsr330-api-java libcdi-api-java libcommons-cli-java
  libcommons-io-java libcommons-lang3-java libcommons-parent-java liberror-prone-java libgeronimo-annotation-1.3-spec-java
  libgeronimo-interceptor-3.0-spec-java libguava-java libguice-java libjansi-java libjsr305-java libmaven-parent-java
  libmaven-resolver-java libmaven-shared-utils-java libmaven3-core-java libplexus-cipher-java libplexus-classworlds-java
  libplexus-component-annotations-java libplexus-interpolation-java libplexus-sec-dispatcher-java libplexus-utils2-java
  libsisu-inject-java libsisu-plexus-java libslf4j-java libwagon-file-java libwagon-http-shaded-java
  libwagon-provider-api-java
Suggested packages:
  libatinject-jsr330-api-java-doc libel-api-java libcommons-io-java-doc libasm-java libcglib-java libjsr305-java-doc
  libmaven-shared-utils-java-doc liblogback-java libplexus-utils2-java-doc junit4 testing libcommons-logging-java
  liblog4j2-java
The following NEW packages will be installed:
  libaopalliance-java libapache-pom-java libatinject-jsr330-api-java libcdi-api-java libcommons-cli-java
  libcommons-io-java libcommons-lang3-java libcommons-parent-java liberror-prone-java libgeronimo-annotation-1.3-spec-java
  libgeronimo-interceptor-3.0-spec-java libguava-java libguice-java libjansi-java libjsr305-java libmaven-parent-java
  libmaven-resolver-java libmaven-shared-utils-java libmaven3-core-java libplexus-cipher-java libplexus-classworlds-java
  libplexus-component-annotations-java libplexus-interpolation-java libplexus-sec-dispatcher-java libplexus-utils2-java
  libsisu-inject-java libsisu-plexus-java libslf4j-java libwagon-file-java libwagon-http-shaded-java
  libwagon-provider-api-java maven
0 upgraded, 32 newly installed, 0 to remove and 4 not upgraded.
Need to get 10.1 MB of archives.
After this operation, 13.1 MB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 libapache-pom-java all 29-2 [5,284 B]
Get:2 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 libatinject-jsr330-api-java all 1.0+ds1-5 [5,348 B]
Get:3 http://pk.archive.ubuntu.com/ubuntu noble/universe amd64 libgeronimo-interceptor-3.0-spec-java all 1.0.1-4fakesync [8,616 B]
```

mvn -version:

verifies that Maven is successfully installed and displays its version.

A terminal window with a dark purple background. The title bar shows a terminal icon, the text 'root@myubuntuvm: /home/ji', and window control buttons. The terminal content shows the command 'mvn -version' being executed, followed by several lines of output: 'Apache Maven 3.8.7', 'Maven home: /usr/share/maven', 'Java version: 21.0.8, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64', 'Default locale: en_US, platform encoding: UTF-8', and 'OS name: "linux", version: "6.8.0-85-generic", arch: "amd64", family: "unix"'. The prompt 'root@myubuntuvm: /home/jamisultani#' is visible at the bottom with a green cursor.

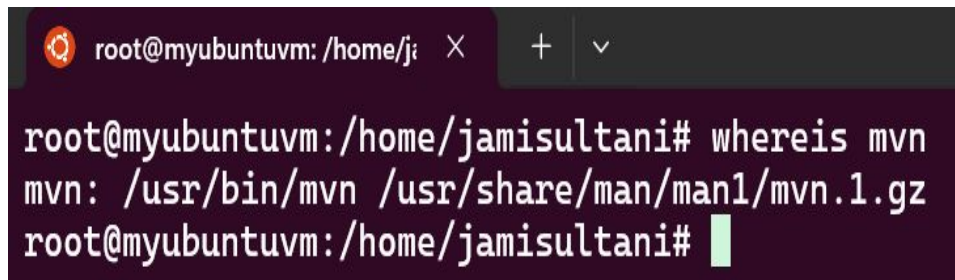
```
root@myubuntuvm: /home/jamisultani# mvn -version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.8, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-85-generic", arch: "amd64", family: "unix"
root@myubuntuvm: /home/jamisultani#
```

◆ Step 8: Configure Maven Globally (Set M2_HOME and PATH).

Commands:

whereis mvn:

locates the Maven installation path.



```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# whereis mvn
mvn: /usr/bin/mvn /usr/share/man/man1/mvn.1.gz
root@myubuntuvvm:/home/jamisultani#
```

echo "export M2_HOME=/usr/share/maven" | sudo tee /etc/profile.d/[maven.sh](#) :

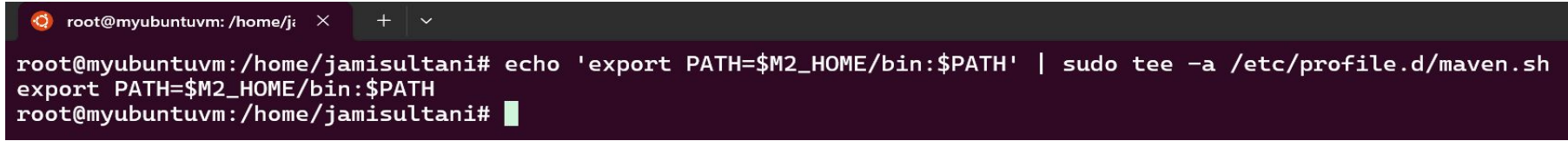
file stores global environment variables , sets the home directory of Maven.



```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# echo "export M2_HOME=/usr/share/maven" | sudo tee /etc/profile.d/maven.sh
export M2_HOME=/usr/share/maven
root@myubuntuvvm:/home/jamisultani#
```

echo 'export PATH=\$M2_HOME/bin:\$PATH' | sudo tee -a /etc/profile.d/[maven.sh](#) :

Adding `$M2_HOME/bin` to `PATH` allows Maven commands to run globally from anywhere.

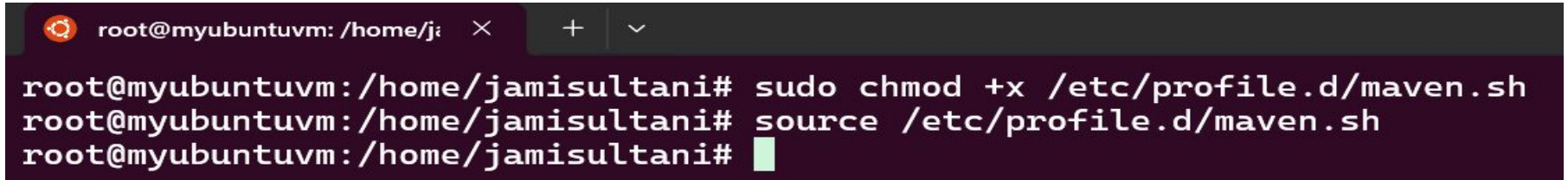
A terminal window with a dark purple background. The title bar shows 'root@myubuntuvvm: /home/ji' with a close button and a dropdown arrow. The terminal text shows the command 'echo 'export PATH=\$M2_HOME/bin:\$PATH' | sudo tee -a /etc/profile.d/maven.sh' being executed, followed by the output 'export PATH=\$M2_HOME/bin:\$PATH' and a new prompt.

```
root@myubuntuvvm: /home/ji × + ▾
root@myubuntuvvm:/home/jamisultani# echo 'export PATH=$M2_HOME/bin:$PATH' | sudo tee -a /etc/profile.d/maven.sh
export PATH=$M2_HOME/bin:$PATH
root@myubuntuvvm:/home/jamisultani#
```

sudo chmod +x /etc/profile.d/[maven.sh](#) :

source /etc/profile.d/[maven.sh](#) :

command applies the changes immediately without rebooting.

A terminal window with a dark purple background. The title bar shows 'root@myubuntuvvm: /home/ji' with a close button and a dropdown arrow. The terminal text shows two commands being executed: 'sudo chmod +x /etc/profile.d/maven.sh' and 'source /etc/profile.d/maven.sh', each followed by a new prompt.

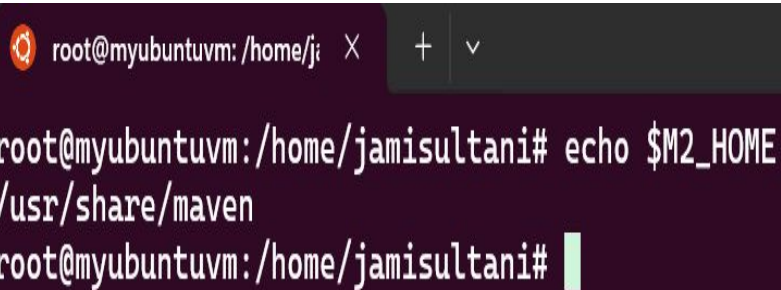
```
root@myubuntuvvm: /home/ji × + ▾
root@myubuntuvvm:/home/jamisultani# sudo chmod +x /etc/profile.d/maven.sh
root@myubuntuvvm:/home/jamisultani# source /etc/profile.d/maven.sh
root@myubuntuvvm:/home/jamisultani#
```

◆ Step 8: Verify Global Configuration.

Commands:

echo \$M2_HOME :

confirms the global Maven home directory.



```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# echo $M2_HOME
/usr/share/maven
root@myubuntuvvm:/home/jamisultani#
```

which mvn:

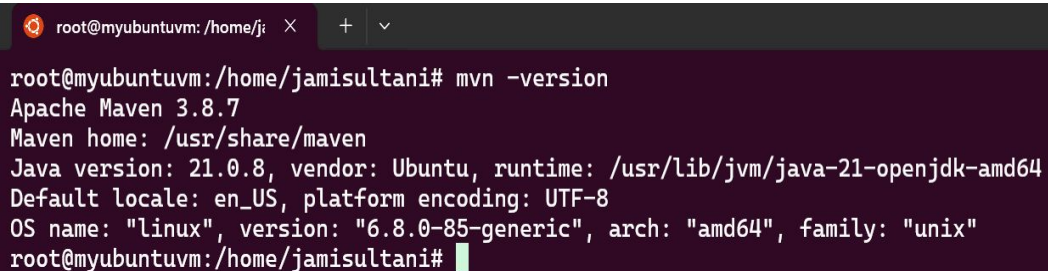
confirms the global Maven home directory.



```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# which mvn
/usr/share/maven/bin/mvn
root@myubuntuvvm:/home/jamisultani#
```

mvn -version:

verifies Maven is working correctly and globally accessible.



```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani# mvn -version
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.8, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.8.0-85-generic", arch: "amd64", family: "unix"
root@myubuntuvvm:/home/jamisultani#
```

◆ Step 9: Install and Configure Jenkins (with GUI Setup).

Commands:

sudo apt update && sudo apt upgrade -y :

This updates and upgrades all system packages to ensure your Ubuntu has the latest dependencies before installing Jenkins.

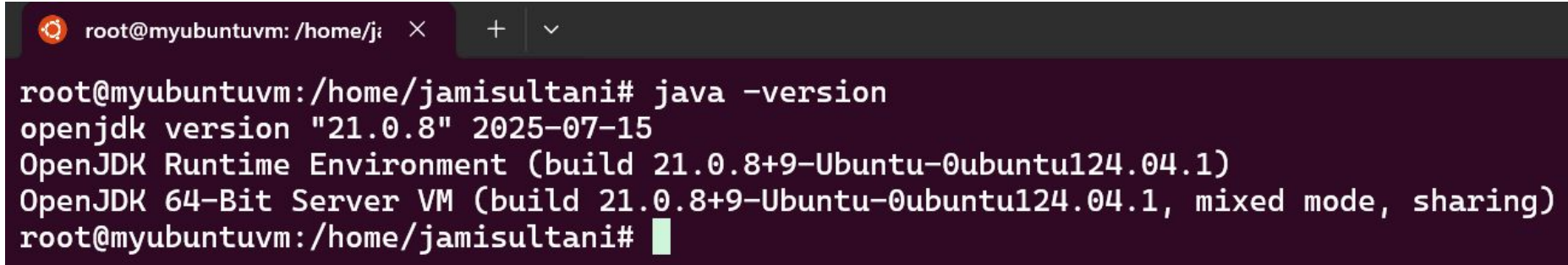
```
root@myubuntuvvm: /home/ji  X  +  v
root@myubuntuvvm:/home/jamisultani# sudo apt update && sudo apt upgrade -y
Hit:1 http://pk.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  bind9-dnswtills bind9-host bind9-libs linux-firmware
4 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
Need to get 538 MB/539 MB of archives.
After this operation, 923 kB of additional disk space will be used.
Ign:1 http://security.ubuntu.com/ubuntu noble-security/main amd64 linux-firmware amd64 20240318.git3b128b60-0ubuntu2.19
Get:1 http://security.ubuntu.com/ubuntu noble-security/main amd64 linux-firmware amd64 20240318.git3b128b60-0ubuntu2.19 [538
MB]
Fetched 538 MB in 5min 5s (1,764 kB/s)
(Reading database ... 153200 files and directories currently installed.)
Preparing to unpack .../bind9-dnswtills_1%3a9.18.39-0ubuntu0.24.04.2_amd64.deb ...
Unpacking bind9-dnswtills (1:9.18.39-0ubuntu0.24.04.2) over (1:9.18.30-0ubuntu0.24.04.2) ...
Preparing to unpack .../bind9-host_1%3a9.18.39-0ubuntu0.24.04.2_amd64.deb ...
Unpacking bind9-host (1:9.18.39-0ubuntu0.24.04.2) over (1:9.18.30-0ubuntu0.24.04.2) ...
Preparing to unpack .../bind9-libs_1%3a9.18.39-0ubuntu0.24.04.2_amd64.deb ...
Unpacking bind9-libs:amd64 (1:9.18.39-0ubuntu0.24.04.2) over (1:9.18.30-0ubuntu0.24.04.2) ...
Preparing to unpack .../linux-firmware_20240318.git3b128b60-0ubuntu2.19_amd64.deb ...
```

◆ Step 10: Install Java (Dependency Check).

Commands:

java -version :

Jenkins needs Java to run. If you've already installed Java (as we did earlier), just verify it.

A terminal window with a dark purple background. The title bar shows 'root@myubuntuvvm: /home/ji' with a close button. The terminal content shows the command 'java -version' being executed, resulting in the following output: 'openjdk version "21.0.8" 2025-07-15', 'OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)', and 'OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)'. The prompt returns to 'root@myubuntuvvm: /home/jamisultani#'.

```
root@myubuntuvvm: /home/ji  ×  +  v
root@myubuntuvvm:/home/jamisultani# java -version
openjdk version "21.0.8" 2025-07-15
OpenJDK Runtime Environment (build 21.0.8+9-Ubuntu-0ubuntu124.04.1)
OpenJDK 64-Bit Server VM (build 21.0.8+9-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)
root@myubuntuvvm:/home/jamisultani#
```

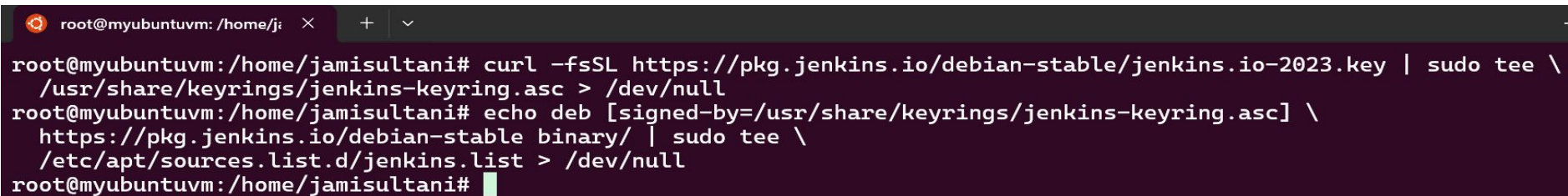
◆ Step 11: Add Jenkins Repository and Key.

Commands:

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

These commands add Jenkins' official GPG key and repository to your system. This ensures you download the latest, secure Jenkins version directly from Jenkins servers.

A terminal window screenshot with a dark background. The window title bar shows 'root@myubuntuvvm: /home/ji' and standard window controls. The terminal displays two commands being executed in sequence. The first command is 'curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \ /usr/share/keyrings/jenkins-keyring.asc > /dev/null'. The second command is 'echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \ https://pkg.jenkins.io/debian-stable binary/ | sudo tee \ /etc/apt/sources.list.d/jenkins.list > /dev/null'. The prompt 'root@myubuntuvvm: /home/jamisultani#' is visible at the start of each line.

```
root@myubuntuvvm: /home/jamisultani# curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc > /dev/null
root@myubuntuvvm: /home/jamisultani# echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
root@myubuntuvvm: /home/jamisultani#
```

◆ Step 12: Install Jenkins.

Commands:

sudo apt install jenkins -y :

Installs Jenkins from the newly added repository.

```
root@myubuntuvvm: /home/j:
root@myubuntuvvm:/home/jamisultani# sudo apt install jenkins -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
jenkins
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 95.0 MB of archives.
After this operation, 95.5 MB of additional disk space will be used.
Get:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.528.1 [95.0 MB]
Ign:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.528.1
Get:1 https://pkg.jenkins.io/debian-stable binary/ jenkins 2.528.1 [95.0 MB]
Fetched 44.0 MB in 5min 32s (132 kB/s)
Selecting previously unselected package jenkins.
(Reading database ... 153455 files and directories currently installed.)
Preparing to unpack .../jenkins_2.528.1_all.deb ...
Unpacking jenkins (2.528.1) ...
Setting up jenkins (2.528.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service → /usr/lib/systemd/system/jenkins.service.
Scanning processes...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
6.8.0-85-generic
Diagnostics:
The currently running kernel version is not the expected kernel version 6.8.0-86-generic.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

No services need to be restarted.
```

◆ Step 13: Start and Enable Jenkins Service.

Commands:

sudo systemctl start jenkins:

runs Jenkins immediately.

sudo systemctl enable jenkins:
ensures Jenkins starts automatically on boot.

```
root@myubuntuvm: /home/ji X + v
root@myubuntuvm:/home/jamisultani# sudo systemctl enable jenkins
Synchronizing state of jenkins.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable jenkins
root@myubuntuvm:/home/jamisultani#
```

sudo systemctl status jenkins:
checks whether Jenkins is active and running.

```
root@myubuntuvm: /home/ji X + v
root@myubuntuvm:/home/jamisultani# sudo systemctl start jenkins
root@myubuntuvm:/home/jamisultani#
```

```
root@myubuntuvm: /home/ji X + v
root@myubuntuvm:/home/jamisultani# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
Active: active (running) since Tue 2025-10-28 17:46:14 UTC; 15h ago
Main PID: 18595 (java)
Tasks: 45 (limit: 9436)
Memory: 621.0M (peak: 626.4M)
CPU: 28.210s
CGroup: /system.slice/jenkins.service
└─18595 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war

Oct 28 17:46:02 myubuntuvm jenkins[18595]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Oct 28 17:46:02 myubuntuvm jenkins[18595]: [LF]>
Oct 28 17:46:02 myubuntuvm jenkins[18595]: [LF]> *****
Oct 28 17:46:02 myubuntuvm jenkins[18595]: [LF]> *****
Oct 28 17:46:14 myubuntuvm jenkins[18595]: 2025-10-28 17:46:14.239+0000 [id=41] INFO jenkins.InitReactorRunner
Oct 28 17:46:14 myubuntuvm jenkins[18595]: 2025-10-28 17:46:14.272+0000 [id=33] INFO hudson.lifecycle.Lifecycle
Oct 28 17:46:14 myubuntuvm system[1]: Started jenkins.service - Jenkins Continuous Integration Server
Oct 28 17:46:20 myubuntuvm jenkins[18595]: 2025-10-28 17:46:20.898+0000 [id=63] INFO h.m.DownloadService$Down
Oct 28 17:46:20 myubuntuvm jenkins[18595]: 2025-10-28 17:46:20.900+0000 [id=63] INFO hudson.util.Retrier$Star
lines 1-20/20 (END)
[1]+ Stopped sudo systemctl status jenkins
root@myubuntuvm:/home/jamisultani#
```

◆ Step 14: Allow Port 8080 in Firewall.

Commands:

sudo ufw enable:

```
root@myubuntuvvm:/home/jami# sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
root@myubuntuvvm:/home/jami#
```

sudo ufw allow 8080:

```
root@myubuntuvvm:/home/jami# sudo ufw allow 8080
Skipping adding existing rule
Skipping adding existing rule (v6)
root@myubuntuvvm:/home/jami#
```

sudo ufw reload:

```
root@myubuntuvvm:/home/jami# sudo ufw reload
Firewall reloaded
root@myubuntuvvm:/home/jami#
```

sudo ufw status:

```
root@myubuntuvvm:/home/jami# sudo ufw status
Status: active
```

To	Action	From
3389/tcp	ALLOW	Anywhere
8080	ALLOW	Anywhere
3389/tcp (v6)	ALLOW	Anywhere (v6)
8080 (v6)	ALLOW	Anywhere (v6)

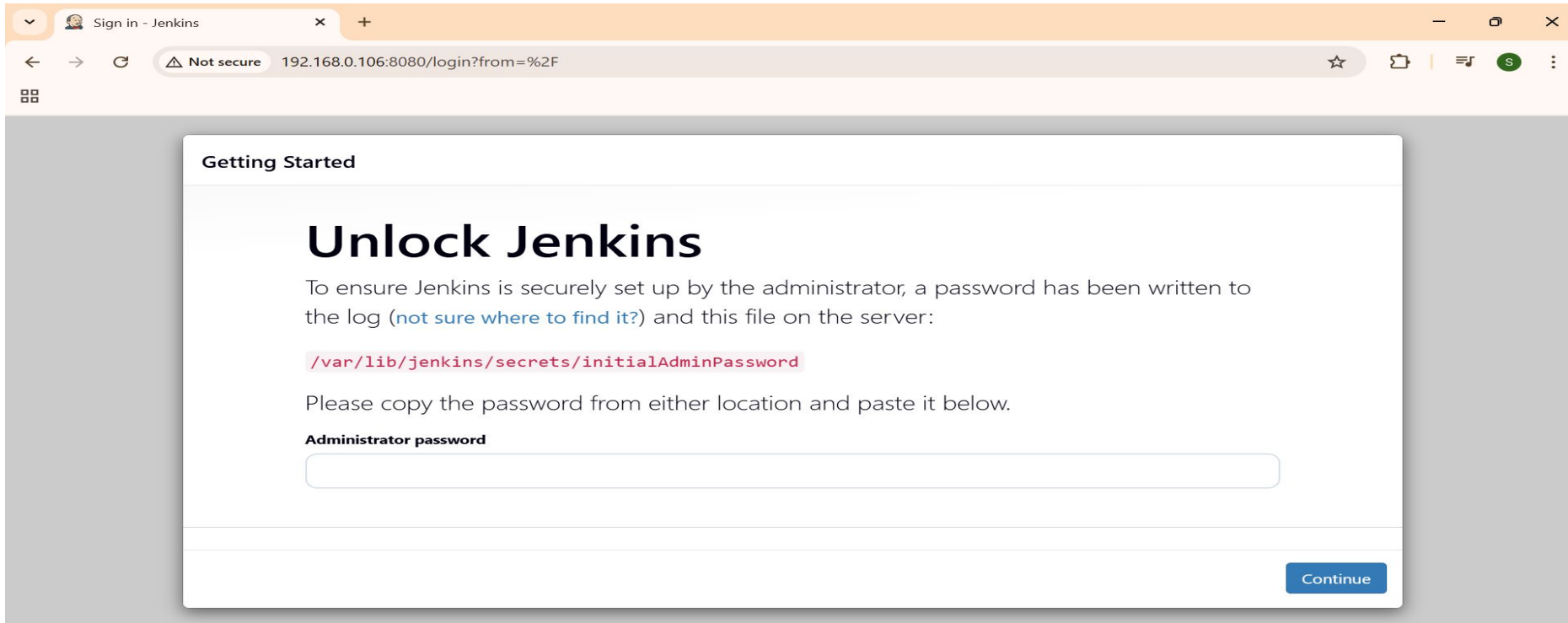
```
root@myubuntuvvm:/home/jami#
```

Jenkins runs on port **8080**, so you need to open it in the firewall to access Jenkins through a web browser.

◆ Step 15: Access Jenkins GUI in Browser.

Commands / URL:

<http://localhost:8080> Or <http://ip:8080>

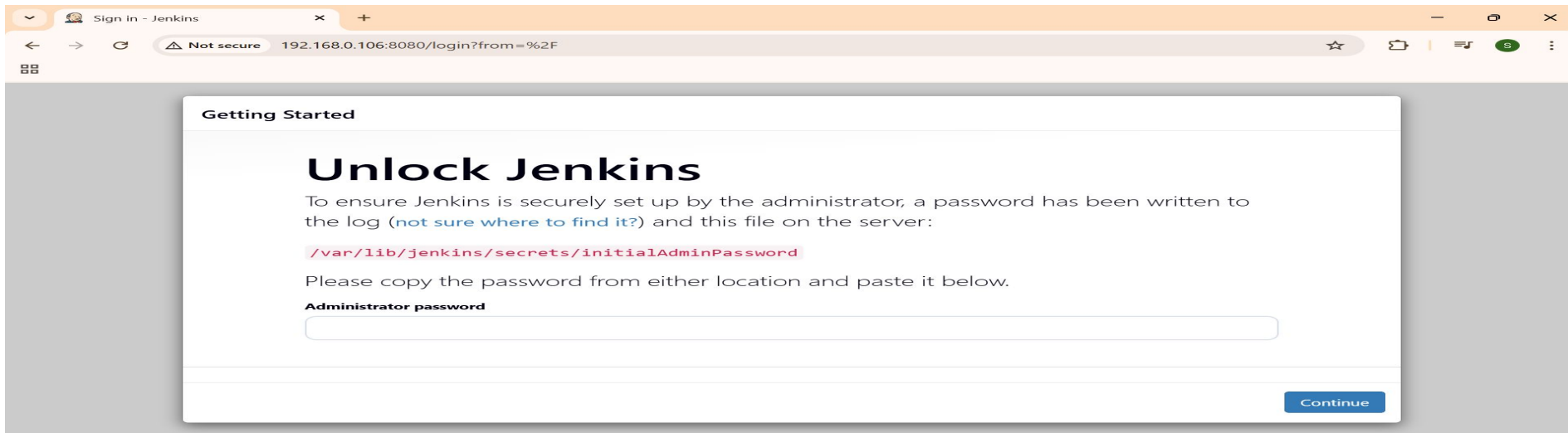


◆ Step 16: Get the Initial Admin Password.

Commands:

sudo cat /var/lib/jenkins/secrets/initialAdminPassword :

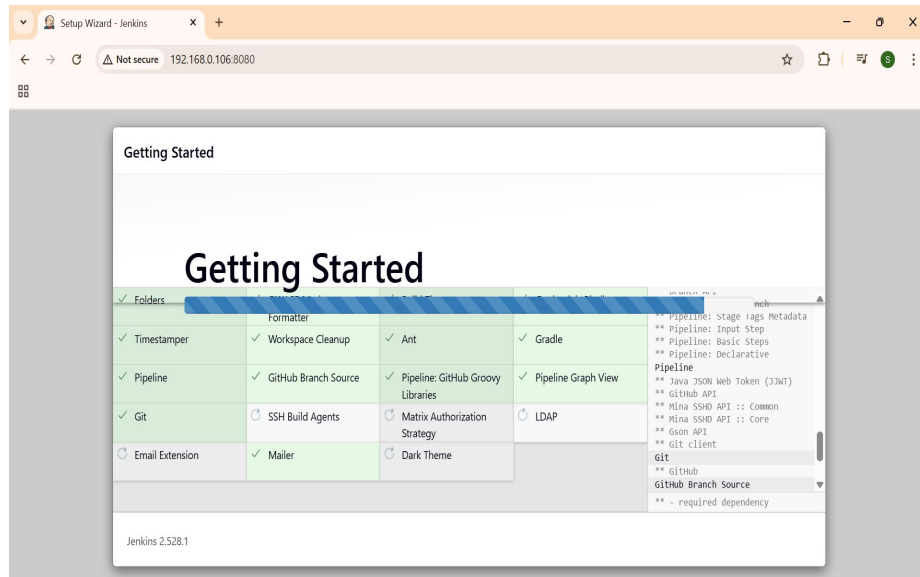
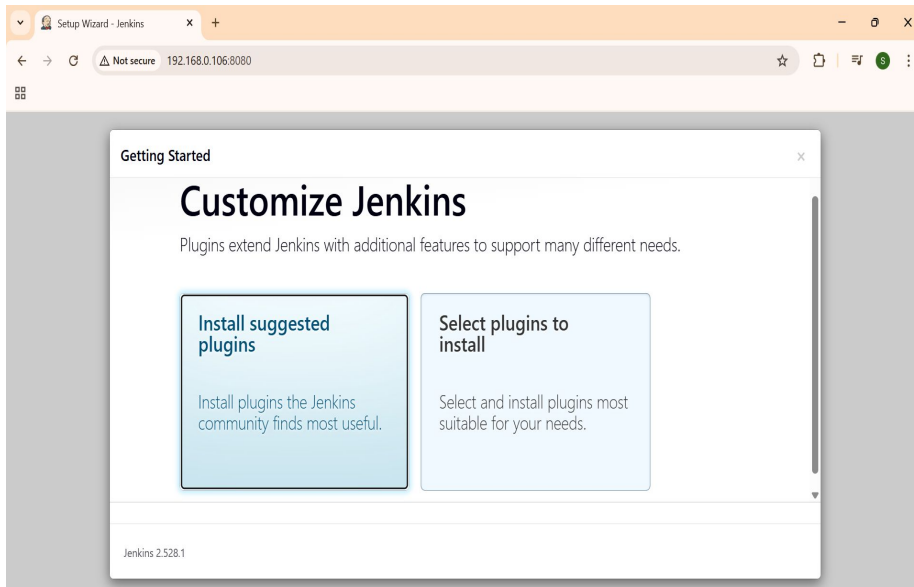
This shows the admin password you'll use to unlock Jenkins in your browser.



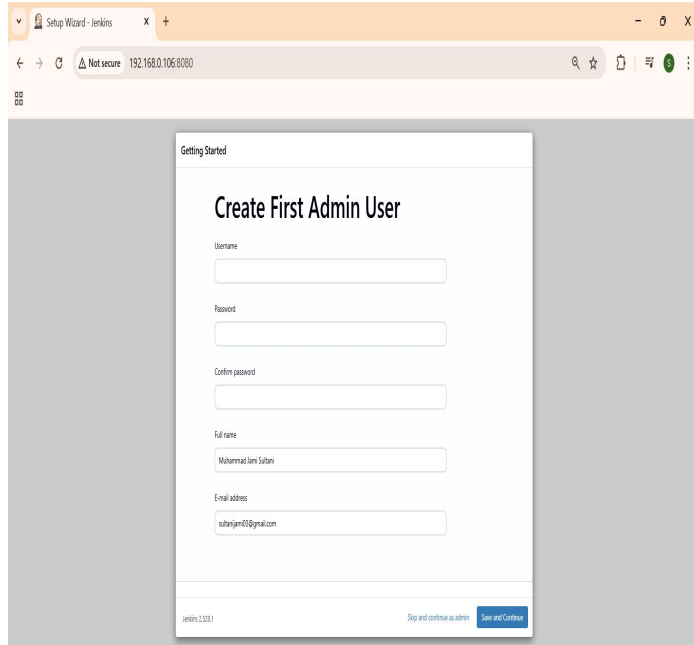
◆ Step 17: Complete Jenkins Setup Wizard.

Steps (in GUI): This step installs plugins and completes Jenkins setup.

1. Copy the password shown in terminal and paste it in browser.
2. Choose “Install suggested plugins.”



3. Create an admin user
(username, password,
full name, email).

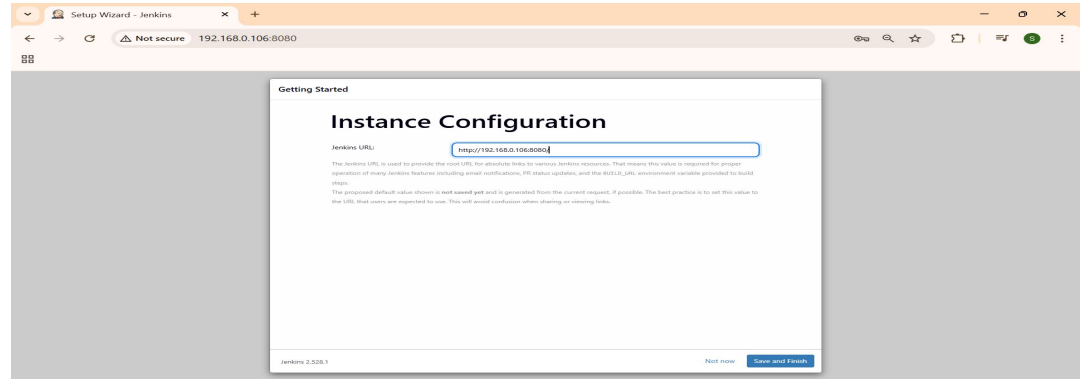


The screenshot shows the 'Create First Admin User' form in the Jenkins Setup Wizard. The form has the following fields and values:

- Username:
- Password:
- Confirm password:
- Full name:
- E-mail address:

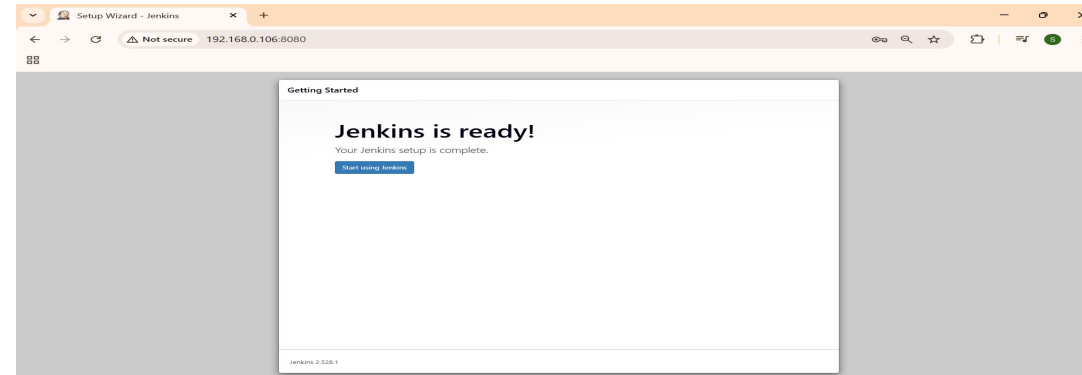
At the bottom, there is a 'Skip and continue as admin' link and a 'Save and Continue' button. The version 'Jenkins 2.326.1' is displayed in the bottom left corner.

4. Set Jenkins URL (keep default)
<http://localhost:8080> Or <http://ip:8080>



The screenshot shows the 'Instance Configuration' screen in the Jenkins Setup Wizard. The 'Jenkins URL' field is pre-filled with <http://192.168.0.106:8080>. Below the field, there is explanatory text about the URL and a 'Save and Finish' button. The version 'Jenkins 2.326.1' is displayed in the bottom left corner.

5. Click “Save and Finish.”



The screenshot shows the 'Jenkins is ready!' screen in the Jenkins Setup Wizard. The text 'Your Jenkins setup is complete.' is displayed, along with a 'Start using Jenkins' button. The version 'Jenkins 2.326.1' is displayed in the bottom left corner.

◆ Step 18: Verify Jenkins Installation.

Commands:

jenkins --version :

sudo systemctl status jenkins :

```
root@myubuntvm:/home/jami# jenkins --version
2.528.1
root@myubuntvm:/home/jami#
```

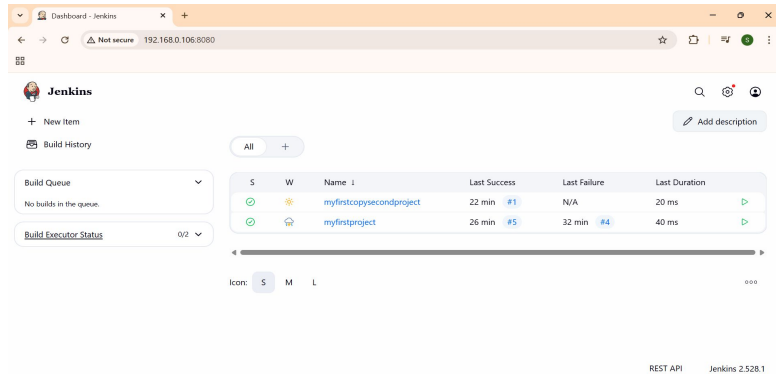
```
root@myubuntvm:/home/jami# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
Active: active (running) since Wed 2025-10-29 10:07:07 UTC; 3h 32min ago
Main PID: 21044 (java)
Tasks: 45 (limit: 9436)
Memory: 786.1M (peak: 787.1M)
CPU: 2min 4.816s
CGroup: /system.slice/jenkins.service
└─21044 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war

Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.839+0000 [id=238] INFO jenkins.InitReactorRunn
Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.839+0000 [id=241] INFO jenkins.InitReactorRunn
Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.920+0000 [id=238] INFO jenkins.InitReactorRunn
Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.926+0000 [id=238] INFO jenkins.InitReactorRunn
Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.966+0000 [id=244] INFO jenkins.InitReactorRunn
Oct 29 10:45:47 myubuntvm jenkins[21044]: 2025-10-29 10:45:47.966+0000 [id=244] INFO jenkins.InitReactorRunn
Oct 29 10:45:48 myubuntvm jenkins[21044]: 2025-10-29 10:45:48.098+0000 [id=239] INFO jenkins.InitReactorRunn
Oct 29 10:45:48 myubuntvm jenkins[21044]: 2025-10-29 10:45:48.103+0000 [id=240] INFO jenkins.InitReactorRunn
Oct 29 10:45:48 myubuntvm jenkins[21044]: 2025-10-29 10:45:48.389+0000 [id=241] INFO jenkins.InitReactorRunn
Oct 29 10:45:48 myubuntvm jenkins[21044]: 2025-10-29 10:45:48.390+0000 [id=90] INFO h.m.UpdateCenter$Comple
```

.Verifies that Jenkins is successfully installed and running on port **8080**.

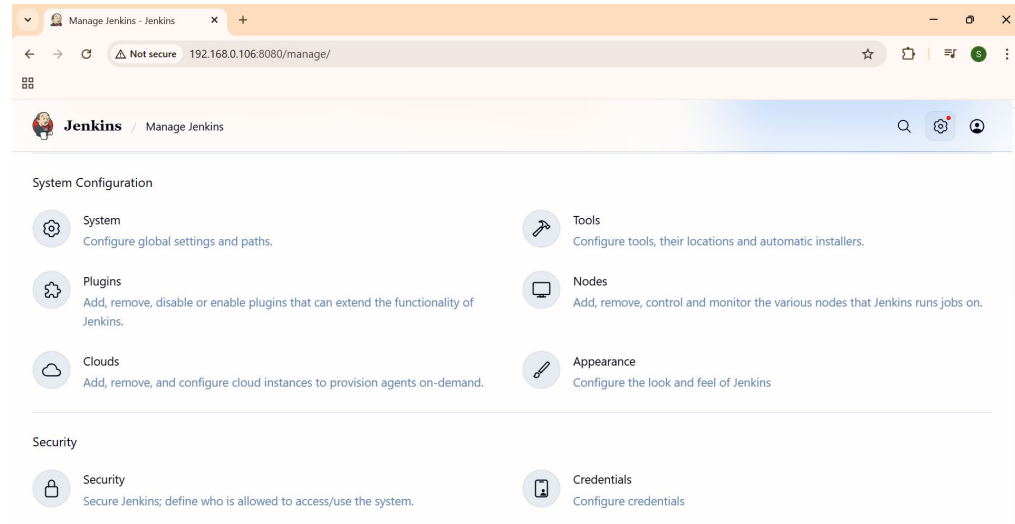
◆ Step 19: Jenkins Global Tools Configuration.

Open Jenkins Dashboard.



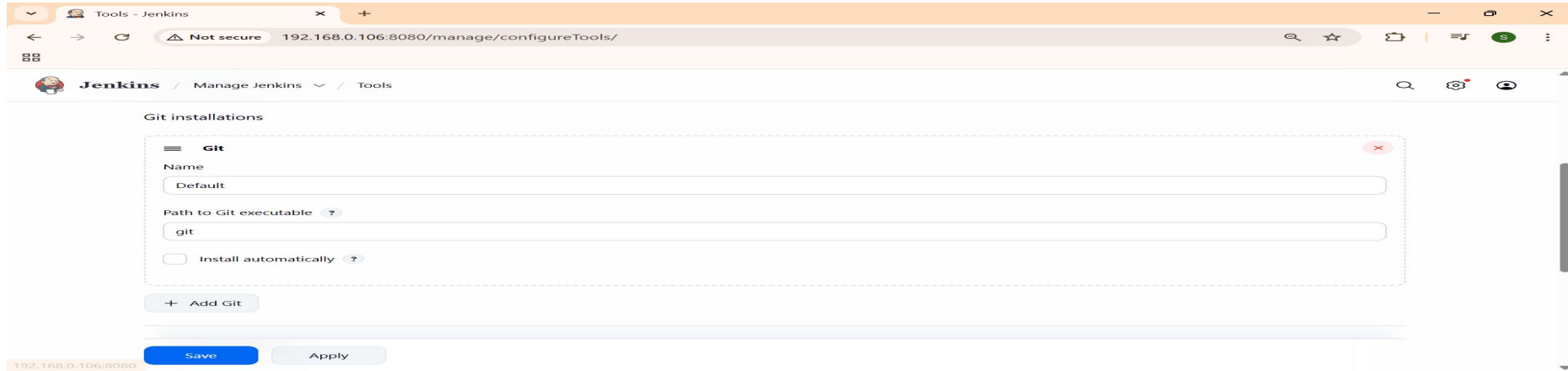
From the Right sidebar, click on:

👉 **Manage Jenkins → Tools → Global Tool Configuration**



◆ Step 20: Configure Git Installation.

1. Scroll down to the **Git** section.
2. Click “**Add Git**”
3. Under **Name**, type: **Git**.
4. Uncheck “*Install automatically*” .
5. In the **Path to Git executable**, paste the Git path you used in your system.



Tools - Jenkins

192.168.0.106:8080/manage/configureTools/

Jenkins / Manage Jenkins / Tools

Git installations

Git

Name

Default

Path to Git executable ?

git

☐ Install automatically ?

+ Add Git

Save Apply

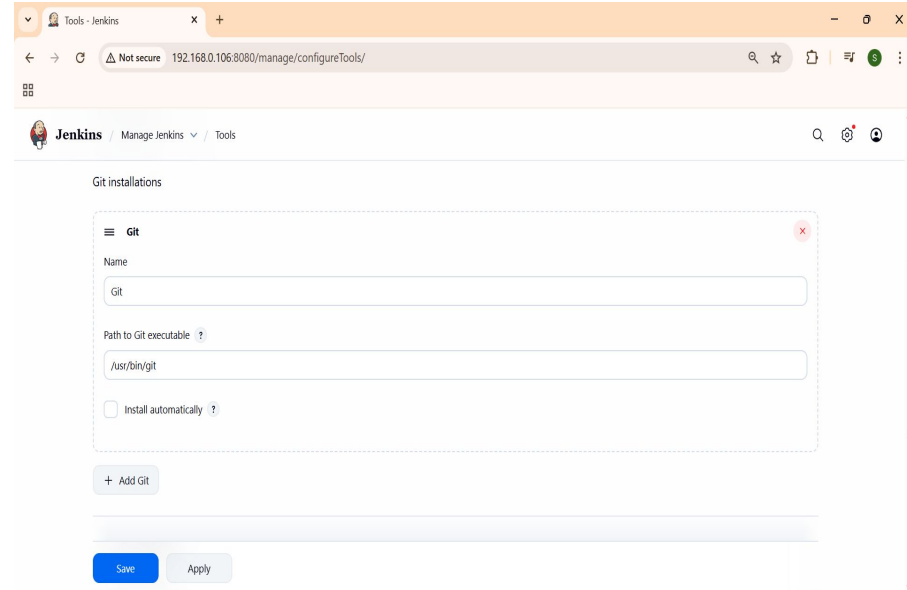
192.168.0.106:8080

👉 On Ubuntu, run this command to get the path:

which git :

```
root@myubuntuvms: /home/jamisultani# which git
/usr/bin/git
root@myubuntuvms: /home/jamisultani#
```

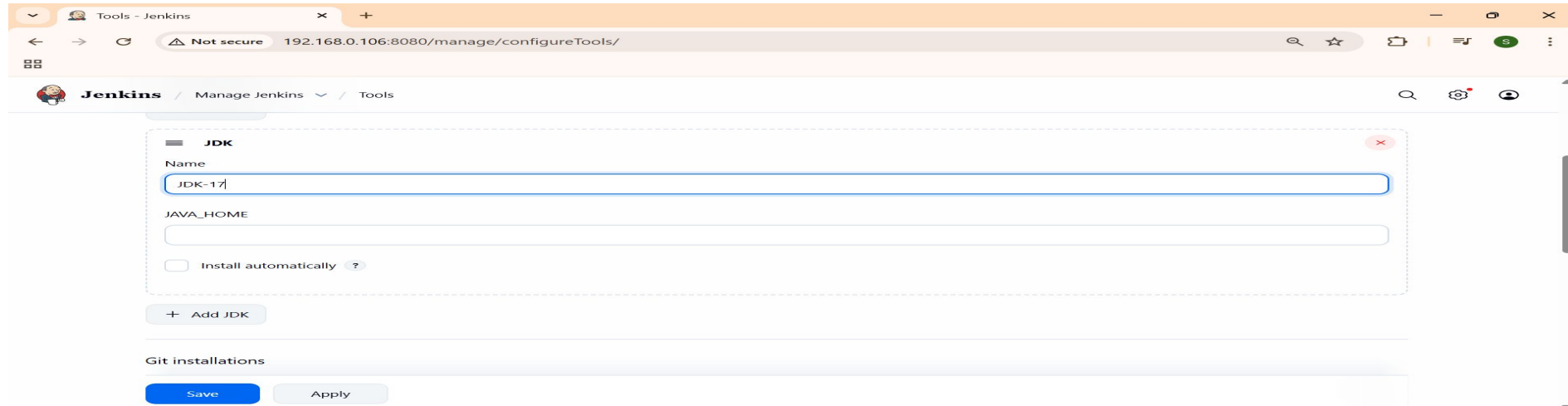
◆ Enter `/usr/bin/git` in Jenkins Git path field.



🧠 **Why?** Jenkins needs this path to execute Git commands like `git clone`, `git fetch`, etc. during CI/CD.

◆ Step 21: Configure JDK (Java).

1. Scroll down to **JDK** section.
2. Click “**Add JDK**”.
3. In the **Name** field, type: **JDK-21**.
4. Uncheck “Install automatically” .
5. Paste the **JDK** installation path.



The screenshot shows the Jenkins web interface for configuring tools. The browser address bar indicates the URL is 192.168.0.106:8080/manage/configureTools/. The page title is 'Tools - Jenkins'. The main content area is titled 'JDK' and contains a form with the following fields:

- Name:** A text input field containing 'JDK-17'.
- JAVA_HOME:** An empty text input field.
- Install automatically:** A checkbox that is currently unchecked, with a help icon (?) to its right.

Below the form is a button labeled '+ Add JDK'. At the bottom of the page, there are two buttons: 'Save' and 'Apply'.

👉 Run this command in terminal to find your Java home path:

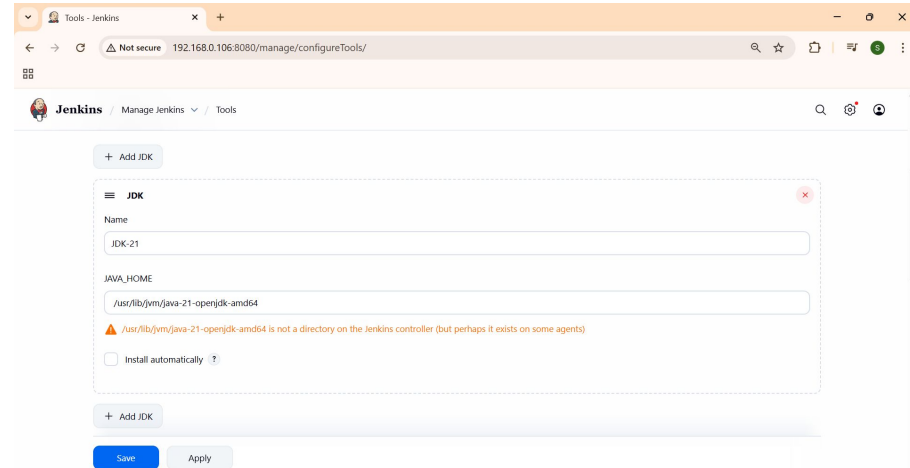
**sudo update-alternatives
--config java :**

```
root@myubuntuvvm:/home/ji X + v
root@myubuntuvvm:/home/jamisultani# sudo update-alternatives --config java
There is 1 choice for the alternative java (providing /usr/bin/java).

  Selection    Path                                          Priority  Status
  -----
* 0            /usr/lib/jvm/java-21-openjdk-amd64/bin/java  2111    auto mode
  1            /usr/lib/jvm/java-21-openjdk-amd64/bin/java  2111    manual mode

Press <enter> to keep the current choice[*], or type selection number: ^Z
[2]+  Stopped                  sudo update-alternatives --config java
root@myubuntuvvm:/home/jamisultani#
```

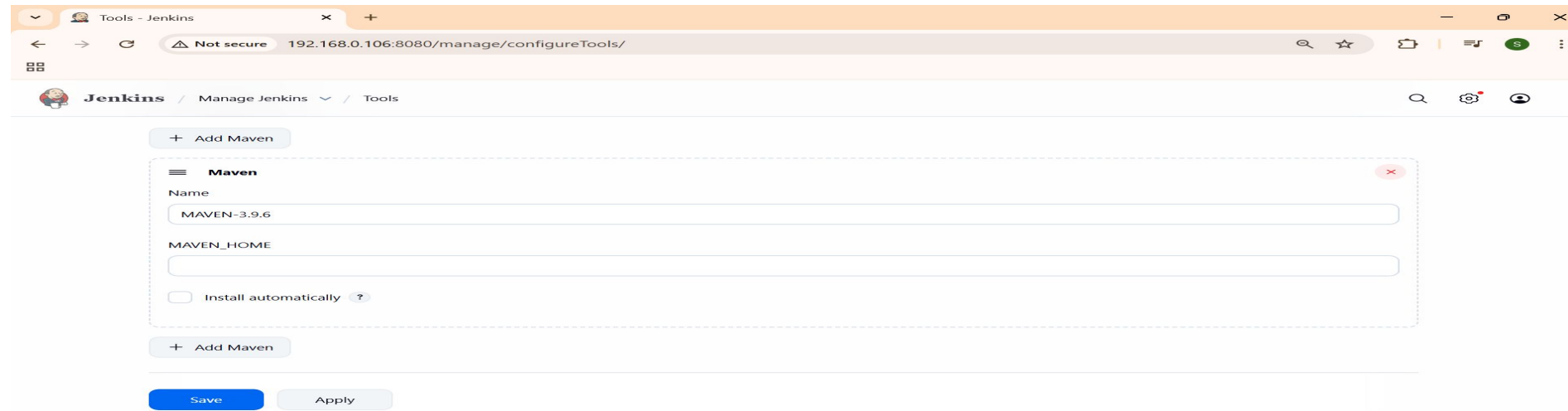
Now, paste the Java home path (without /bin/java):



🧠 **Why?** Jenkins uses JDK to run Java-based builds (like Maven, Gradle, or Jenkins internal jobs).

◆ Step 22: Configure Maven.

1. Scroll down to **Maven** section.
2. Click “**Add Maven**”
3. Type name: **Maven-3.9.6**
4. Uncheck “*Install automatically*” 
5. Paste your **Maven** installation path.



The screenshot shows the Jenkins web interface at the URL `192.168.0.106:8080/manage/configureTools/`. The page title is "Tools - Jenkins". The breadcrumb navigation shows "Manage Jenkins" > "Tools".

The "Add Maven" form is displayed with the following fields and options:

- Name:** `MAVEN-3.9.6`
- MAVEN_HOME:** (Empty text field)
- Install automatically:** ☐ (Unchecked)

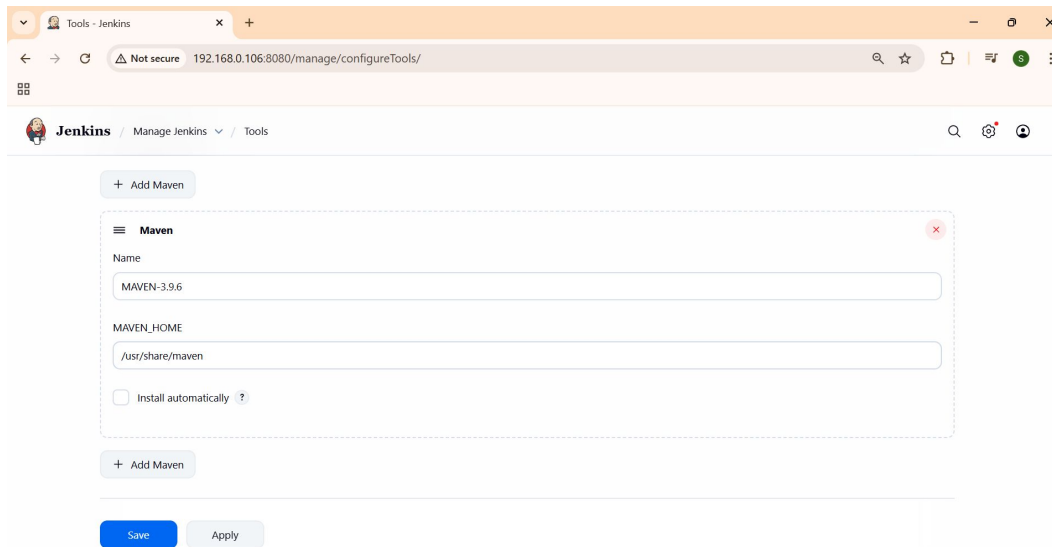
At the bottom of the form, there are "Save" and "Apply" buttons.

👉 To check Maven path run this command:

which mvn:

```
root@myubuntuvms: /home/ji X + v
root@myubuntuvms:/home/jamisultani# which mvn
/usr/bin/mvn
root@myubuntuvms:/home/jamisultani#
```

Now paste the **Maven home directory** (without `/bin/mvn`):



The screenshot shows the Jenkins web interface for configuring tools. The browser address bar indicates the URL is `192.168.0.106:8080/manage/configureTools/`. The page title is 'Jenkins / Manage Jenkins / Tools'. There is a '+ Add Maven' button at the top. Below it, a dashed box contains the 'Maven' configuration section. It has a 'Name' field with the value 'MAVEN-3.9.6', a 'MAVEN_HOME' field with the value '/usr/share/maven', and an unchecked checkbox for 'Install automatically'. Below the dashed box is another '+ Add Maven' button. At the bottom of the page are 'Save' and 'Apply' buttons.

🧠 **Why?** Maven is used by Jenkins to build, compile, and package Java applications automatically during CI.

◆ Step 23: Save Configuration.

1. Scroll down to the bottom of the page.
2. Click on **Save** 
3. Jenkins will now remember all your configured tool paths globally.





Environment Setup (Before Starting the Project).

Before starting the **Jenkins** building & testing **CI/CD** project, all required tools and configurations were properly set up and verified in the Ubuntu (Linux) environment.

This ensured a smooth workflow for continuous integration.



Pre-Configuration Summary:

1. **Virtual Machine (VM)** — Ubuntu OS was launched and configured for DevOps practice.
2. **Java (JDK 21)** — Installed and configured system-wide for Maven and Jenkins compatibility.
3. **Apache Maven (3.9.6)** — Installed and verified using `mvn -v` command.

4. Git — Installed, configured (`git config --global user.name / user.email`), and linked with GitHub.

5. Jenkins — Installed, unlocked, and fully configured with required plugins (Git + Maven + Pipeline).

6. Jenkins Tools Path — JDK and Maven were added inside Jenkins → *Manage Jenkins* → *Global Tool Configuration*.

7. GitHub Account — Active repository access verified for source code management.

8. Network Connectivity — Verified SSH/HTTP connection between Jenkins server and GitHub.

9. Basic Testing — A sample Jenkins freestyle job was successfully executed to verify the setup.



Next Phase: Start Project Development.

Now that the entire DevOps environment is configured and working correctly,

we are ready to **create and automate a Java Maven project using Jenkins CI.**

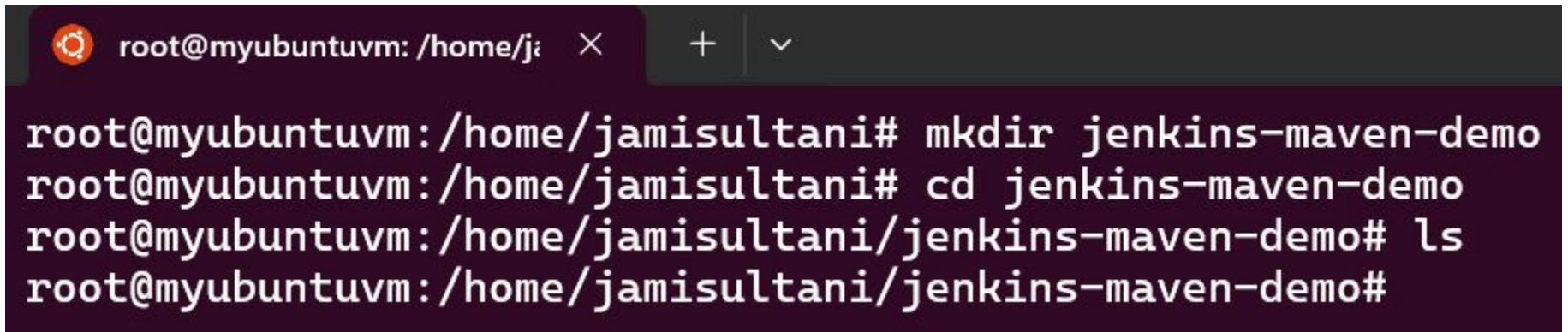
◆ Step 24: Create Project Directory.

Commands:

mkdir jenkins-maven-demo :

cd jenkins-maven-demo :

This command creates a new directory and navigates into it.



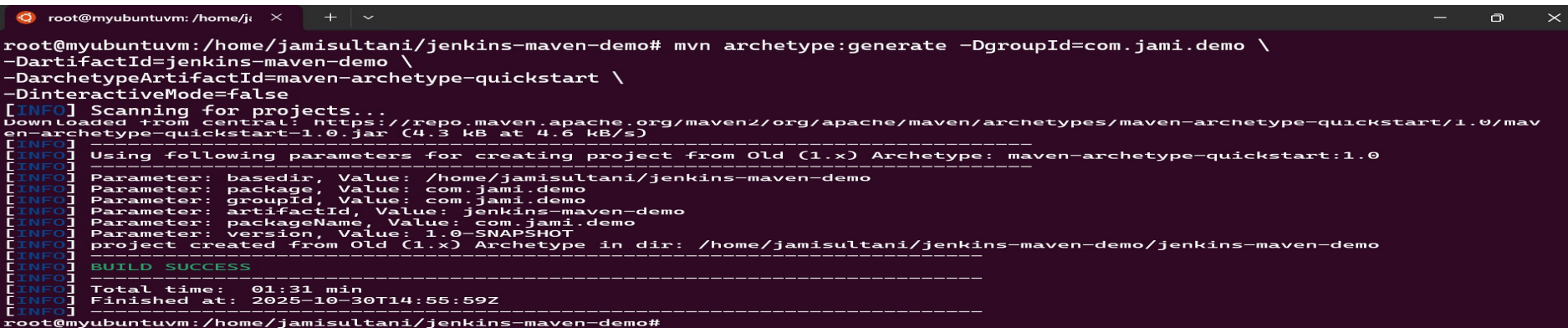
```
root@myubuntuvvm: /home/jamisultani# mkdir jenkins-maven-demo
root@myubuntuvvm: /home/jamisultani# cd jenkins-maven-demo
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo# ls
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo#
```

◆ Step 25: Generate Maven Project Template.

Commands:

```
mvn archetype:generate -DgroupId=com.jami.demo \
-DArtifactId=jenkins-maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

This command generates a standard Java Maven project structure with `pom.xml`, `src`, and `test` folders.

A terminal window with a dark background and light-colored text. The title bar shows 'root@myubuntuvvm: /home/ji'. The command prompt is 'root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo#'. The command entered is 'mvn archetype:generate -DgroupId=com.jami.demo \ -DArtifactId=jenkins-maven-demo \ -DarchetypeArtifactId=maven-archetype-quickstart \ -DinteractiveMode=false'. The output shows the Maven process downloading the archetype from the central repository, scanning for projects, and then using the 'maven-archetype-quickstart:1.0' archetype to create a new project. It lists the parameters used: basedir, package, groupId, artifactId, packageName, and version. The project is created in the directory '/home/jamisultani/jenkins-maven-demo/jenkins-maven-demo'. The process ends with 'BUILD SUCCESS' and shows the total time and finish time.

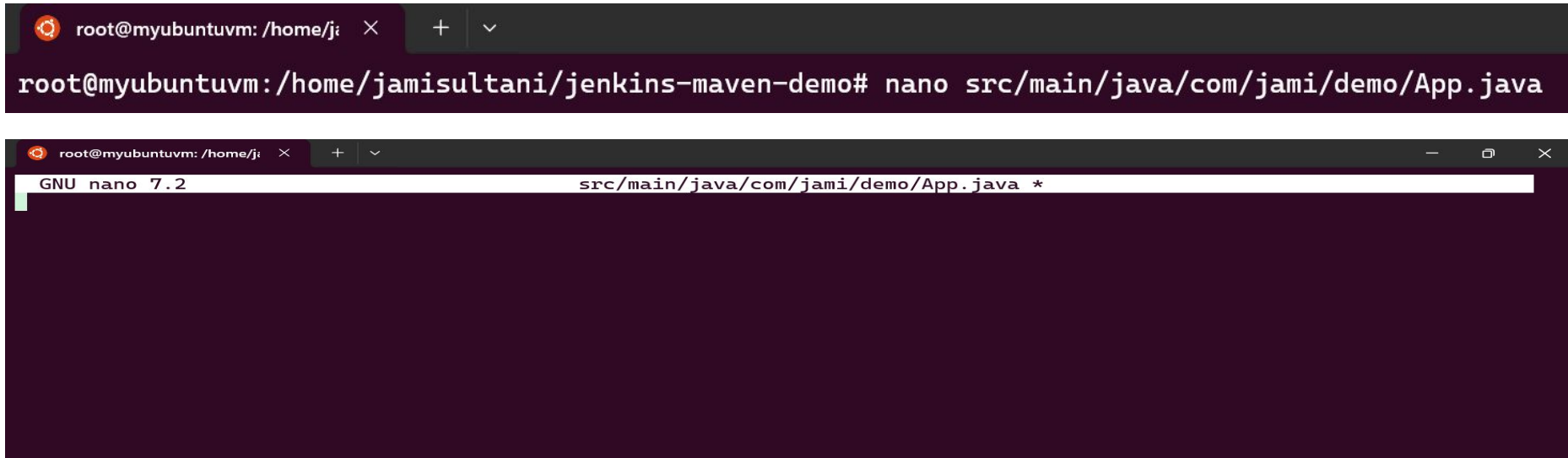
```
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo# mvn archetype:generate -DgroupId=com.jami.demo \
-DArtifactId=jenkins-maven-demo \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
[INFO] Scanning for projects...
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.jar (4.3 kB at 4.6 kB/s)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] Parameter: basedir, Value: /home/jamisultani/jenkins-maven-demo
[INFO] Parameter: package, Value: com.jami.demo
[INFO] Parameter: groupId, Value: com.jami.demo
[INFO] Parameter: artifactId, Value: jenkins-maven-demo
[INFO] Parameter: packageName, Value: com.jami.demo
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/jamisultani/jenkins-maven-demo/jenkins-maven-demo
[INFO] BUILD SUCCESS
[INFO] Total time: 01:31 min
[INFO] Finished at: 2025-10-30T14:55:59Z
[INFO]
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo#
```

◆ Step 26: Update App.java File.

Commands:

Open File:

nano src/main/java/com/jami/demo/App.java :

The image shows two screenshots of a terminal environment. The top screenshot shows a terminal window with a dark purple background. The prompt is 'root@myubuntuvvm: /home/ji'. The command 'nano src/main/java/com/jami/demo/App.java' has been entered. The bottom screenshot shows the nano text editor interface. The title bar at the top reads 'GNU nano 7.2' and 'src/main/java/com/jami/demo/App.java *'. The editor area is currently empty, with a green cursor at the start of the first line.

Replace with this code

```
package com.jami.demo;
```

```
// ✅ Jenkins Maven Demo Project  
// Author: Muhammad Jami Sultani  
// Organization: Rhombix Technologies  
// Version: 1.0.0  
// Date: October 2025
```

```
import java.util.*;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("🚀 Starting Jenkins Maven Demo Project...");  
        System.out.println("👤 Developer: Muhammad Jami Sultani");  
        System.out.println("🏢 Company: Rhombix Technologies");  
        System.out.println("📅 Date: October 2025");  
        System.out.println("-----");
```

```
        greetUser("Jenkins");  
        performCalculations();  
        showDevOpsTools();  
        motivationQuote();
```

```
        System.out.println("-----");  
        System.out.println("✅ CI/CD pipeline test successful!");  
        System.out.println("🎉 Build executed and verified through Jenkins!");
```

```
    }
```

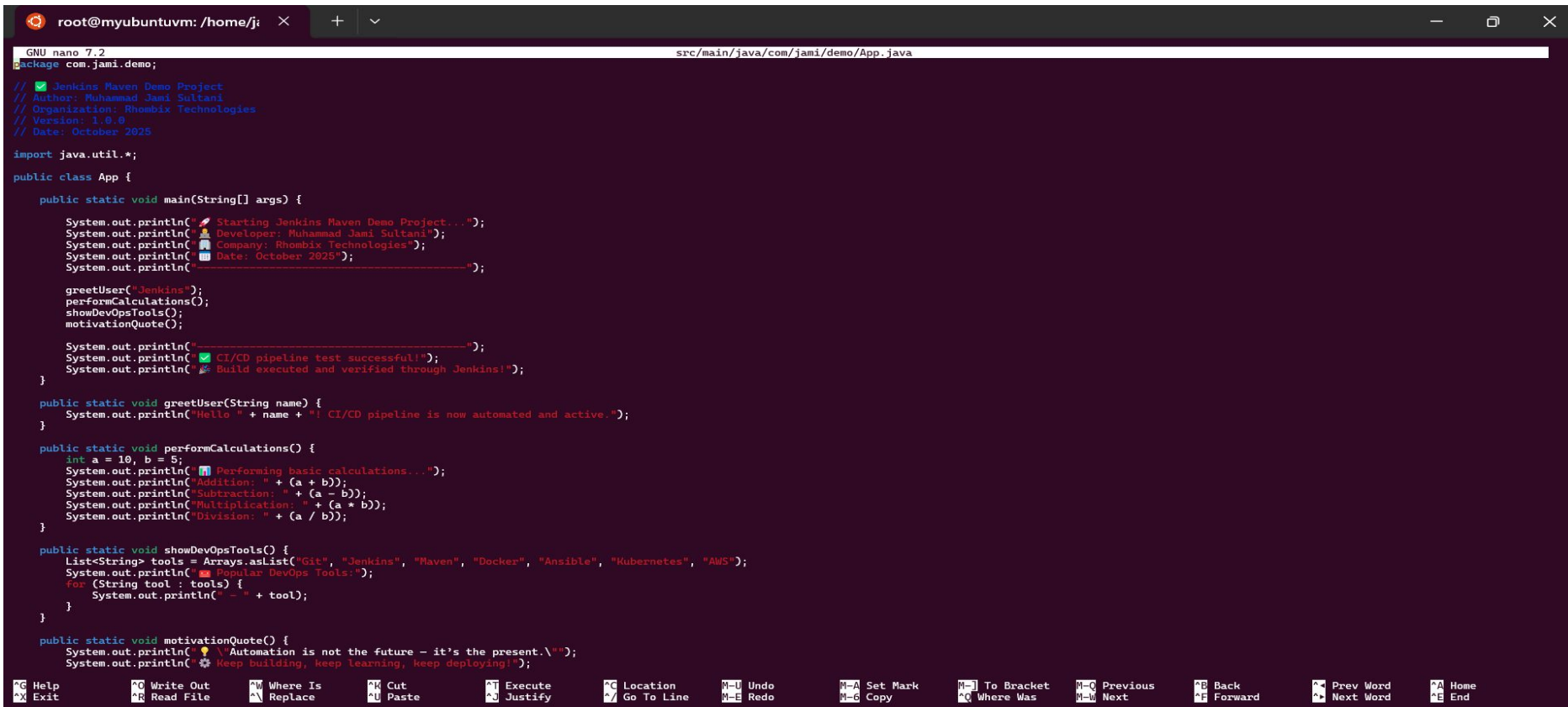
```
        public static void greetUser(String name) {  
            System.out.println("Hello " + name + "! CI/CD pipeline is now automated and active.");  
        }
```

```
        public static void performCalculations() {  
            int a = 10, b = 5;  
            System.out.println("📊 Performing basic calculations...");  
            System.out.println("Addition: " + (a + b));  
            System.out.println("Subtraction: " + (a - b));  
            System.out.println("Multiplication: " + (a * b));  
            System.out.println("Division: " + (a / b));  
        }
```

```
        public static void showDevOpsTools() {  
            List<String> tools = Arrays.asList("Git", "Jenkins", "Maven", "Docker",  
                                                "Ansible", "Kubernetes", "AWS");  
            System.out.println("📦 Popular DevOps Tools:");  
            for (String tool : tools) {  
                System.out.println("- " + tool);  
            }  
        }
```

```
        public static void motivationQuote() {  
            System.out.println("💡 \"Automation is not the future — it's the present.\"");  
            System.out.println("🔧 Keep building, keep learning, keep deploying!");  
        }  
    }
```

This Java program prints developer info, performs basic arithmetic, lists DevOps tools, and prints motivational messages.



The screenshot shows a terminal window with the nano text editor open. The editor is editing a file named `App.java` located at `src/main/java/com/jami/demo/App.java`. The code is a Java program that prints developer information, performs basic arithmetic, lists DevOps tools, and prints motivational messages. The code is as follows:

```
package com.jami.demo;

// 🟢 Jenkins Maven Demo Project
// Author: Muhammad Jami Sultani
// Organization: Rhombix Technologies
// Version: 1.0.0
// Date: October 2025

import java.util.*;

public class App {

    public static void main(String[] args) {

        System.out.println("🚀 Starting Jenkins Maven Demo Project...");
        System.out.println("👤 Developer: Muhammad Jami Sultani");
        System.out.println("🏢 Company: Rhombix Technologies");
        System.out.println("📅 Date: October 2025");
        System.out.println("-----");

        greetUser("Jenkins");
        performCalculations();
        showDevOpsTools();
        motivationQuote();

        System.out.println("-----");
        System.out.println("✅ CI/CD pipeline test successful!");
        System.out.println("🚀 Build executed and verified through Jenkins!");
    }

    public static void greetUser(String name) {
        System.out.println("Hello " + name + "! CI/CD pipeline is now automated and active.");
    }

    public static void performCalculations() {
        int a = 10, b = 8;
        System.out.println("🧮 Performing basic calculations...");
        System.out.println("Addition: " + (a + b));
        System.out.println("Subtraction: " + (a - b));
        System.out.println("Multiplication: " + (a * b));
        System.out.println("Division: " + (a / b));
    }

    public static void showDevOpsTools() {
        List<String> tools = Arrays.asList("Git", "Jenkins", "Maven", "Docker", "Ansible", "Kubernetes", "AWS");
        System.out.println("🔧 Popular DevOps Tools:");
        for (String tool : tools) {
            System.out.println("• " + tool);
        }
    }

    public static void motivationQuote() {
        System.out.println("💡 \"Automation is not the future - it's the present.\");");
        System.out.println("🔥 Keep building, keep learning, keep deploying!");
    }
}
```

The terminal window has a dark theme. The nano editor's status bar at the bottom shows various keyboard shortcuts for editing and navigation.

◆ Step 27: Create Jenkinsfile (Pipeline Definition).

Commands:

nano Jenkinsfile :

```
root@myubuntuvms: /home/jf: X + v
root@myubuntuvms: /home/jamisultani/jenkins-maven-demo# nano Jenkinsfile
```

```
root@myubuntuvms: /home/jf: X + v - 0 X
GNU nano 7.2 Jenkinsfile
```

```
pipeline {
  agent any
  tools {
    maven 'Maven-3.9.6'
    jdk 'JDK-21'
  }
  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build & Test') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
  post {
    success {
      echo '✅ Build Success — CI Pipeline Completed!'
    }
    failure {
      echo '❌ Build Failed — Check Console Output.'
    }
  }
}
```

```
root@myubuntuvvm: /home/ji × + ▾
GNU nano 7.2 Jenkinsfile
pipeline {
  agent any
  tools {
    maven 'Maven-3.9.6'
    jdk 'JDK-21'
  }
  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }
    stage('Build & Test') {
      steps {
        sh 'mvn clean install'
      }
    }
  }
  post {
    success {
      echo '✅ Build Success - CI Pipeline Completed!'
    }
    failure {
      echo '❌ Build Failed - Check Console Output.'
    }
  }
}
```

This defines the Jenkins pipeline — checkout code from GitHub, build using Maven, and show build result.

◆ Step 28: Run Maven Build & Test Locally.

Commands:

mvn clean install :

Compiles and tests your project to confirm everything works fine before integrating with Jenkins.

A terminal window with a dark purple background and white text. The window title is 'root@myubuntuvm: /home/ji'. The command 'mvn clean install' has been executed. The output shows Maven scanning for projects, building 'jenkins-maven-demo 1.0-SNAPSHOT', downloading dependencies from central, and installing the resulting JAR and POM files into the local repository. The build is successful, with a total time of 01:36 min and a finish time of 2025-10-30T15:57:51Z.

```
root@myubuntuvm: /home/jamisultani/jenkins-maven-demo/jenkins-maven-demo# mvn clean install
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.jami.demo:jenkins-maven-demo >-----
[INFO] Building jenkins-maven-demo 1.0-SNAPSHOT
[INFO] [ jar ]-----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-compiler-plugin/3.11.0/
maven-compiler-plugin-3.11.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0
.5.jar (230 kB at 261 kB/s)
[INFO] Installing /home/jamisultani/jenkins-maven-demo/jenkins-maven-demo/target/jenkins-maven-demo-1.0-SNAPSHOT.jar
to /root/.m2/repository/com/jami/demo/jenkins-maven-demo/1.0-SNAPSHOT/jenkins-maven-demo-1.0-SNAPSHOT.jar
[INFO] Installing /home/jamisultani/jenkins-maven-demo/jenkins-maven-demo/pom.xml to /root/.m2/repository/com/jami/de
mo/jenkins-maven-demo/1.0-SNAPSHOT/jenkins-maven-demo-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:36 min
[INFO] Finished at: 2025-10-30T15:57:51Z
[INFO] -----
root@myubuntuvm: /home/jamisultani/jenkins-maven-demo/jenkins-maven-demo#
```

◆ Step 29: Create a New GitHub Repository.

1. Go to <https://github.com>
2. Click **New Repository**
3. Fill details:
 - Name: **jenkins-maven-demo**
 - Description: “Sample Java Maven project for Jenkins CI/CD pipeline demo”
 - Visibility: Public
4. Click **Create Repository**

New repository

github.com/new

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

General

Owner * jamisutani Repository name * jenkins-maven-demo
jenkins-maven-demo is available.

Great repository names are short and memorable. How about [turbo-doodle?](#)

Description
Sample Java Maven project for Jenkins CI/CD pipeline demo
57 / 350 characters

Configuration

Choose visibility *
Choose who can see and commit to this repository. Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#) Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#) No .gitignore

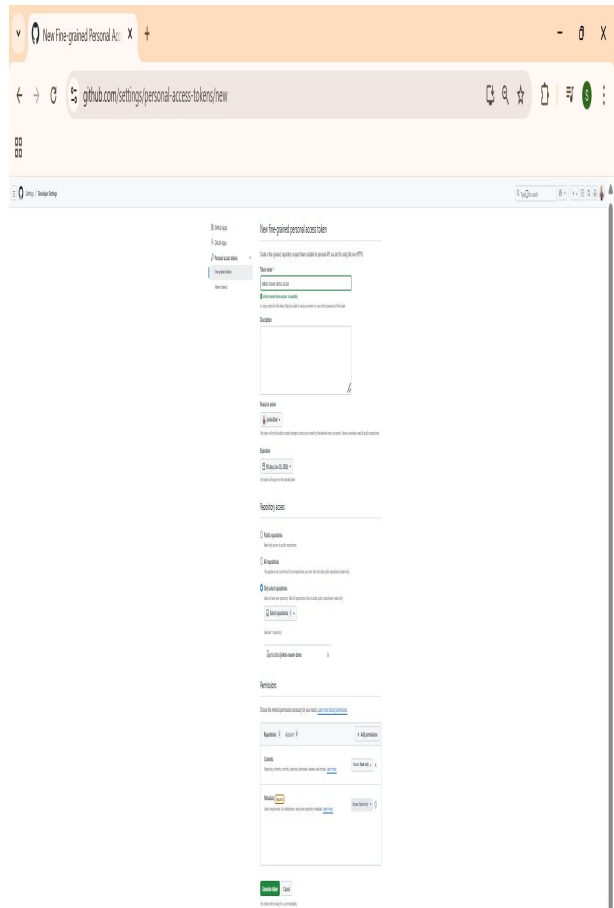
Add license
Licenses explain how others can use your code. [About licenses](#) No license

Create repository

◆ Step 30: Create a Personal Access Token (PAT).

1. Go to <https://github.com/settings/tokens>
2. Click **Fine-grained tokens** → **Generate new token**
3. Set details:
 - Token name: `jenkins-maven-demo-access`
 - Expiration: 90 days
 - Repository access: `jenkins-maven-demo`
4. Enable Permissions:
 - Contents → Read & Write
 - Metadata → Read-only
5. Click **Generate Token**:

Copy token immediately (e.g.
`ghp_XYZabc1234exampleTokennhi`



◆ Step 31: Initialize Git and Push to GitHub.

Commands:

git init :

Initializes a new Git repository in your project folder.

git add . :

Adds all the files in your current directory to the Git staging area.

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani/jenkins-maven-demo# git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/jamisultani/jenkins-maven-demo/.git/
root@myubuntuvvm:/home/jamisultani/jenkins-maven-demo#
```

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm:/home/jamisultani/jenkins-maven-demo# git add .
root@myubuntuvvm:/home/jamisultani/jenkins-maven-demo#
```

git commit -m "Initial commit - Jenkins Maven demo project" :

Creates a new commit and saves your staged changes with a message.

```
root@myubuntuv: /home/ji X + v
root@myubuntuv:/home/jamisultani/jenkins-maven-demo# git commit -m "Initial commit - Jenkins Maven demo project"
[master (root-commit) 07eee69] Initial commit - Jenkins Maven demo project
13 files changed, 163 insertions(+)
create mode 100644 pom.xml
create mode 100644 src/main/java/com/jami/demo/App.java
create mode 100644 src/test/java/com/jami/demo/AppTest.java
create mode 100644 target/classes/com/jami/demo/App.class
create mode 100644 target/jenkins-maven-demo-1.0-SNAPSHOT.jar
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.jami.demo.AppTest.xml
create mode 100644 target/surefire-reports/com.jami.demo.AppTest.txt
create mode 100644 target/test-classes/com/jami/demo/AppTest.class
root@myubuntuv:/home/jamisultani/jenkins-maven-demo#
```

git branch -M main :

Renames the current branch to **main**, which is now GitHub's default branch name.

```
root@myubuntuv:/home/ji X + v
root@myubuntuv:/home/jamisultani/jenkins-maven-demo# git branch -M main
root@myubuntuv:/home/jamisultani/jenkins-maven-demo#
```

git remote add origin

<https://github.com/jamisultani/jenkins-maven-demo.git>

Connects your local Git repository with the remote repository on GitHub.

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo# git remote add origin
https://github.com/jamisultani/jenkins-maven-demo.git
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo# █
```

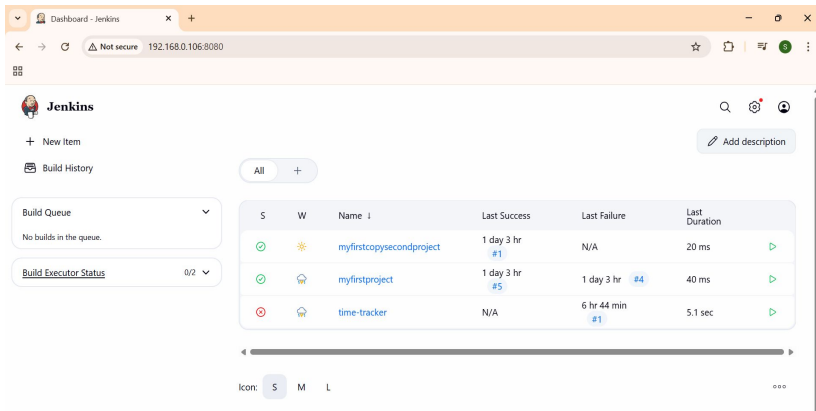
git push -u origin main :

Pushes your local commits to the GitHub repository's **main** branch.

```
root@myubuntuvvm: /home/ji X + v
root@myubuntuvvm: /home/jamisultani/jenkins-maven-demo# git push -u origin main
Username for 'https://github.com': jamisultani
Password for 'https://jamisultani@github.com':
Enumerating objects: 43, done.
Counting objects: 100% (43/43), done.
Delta compression using up to 4 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (43/43), 6.26 KiB | 320.00 KiB/s, done.
Total 43 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/jamisultani/jenkins-maven-demo.git
* [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

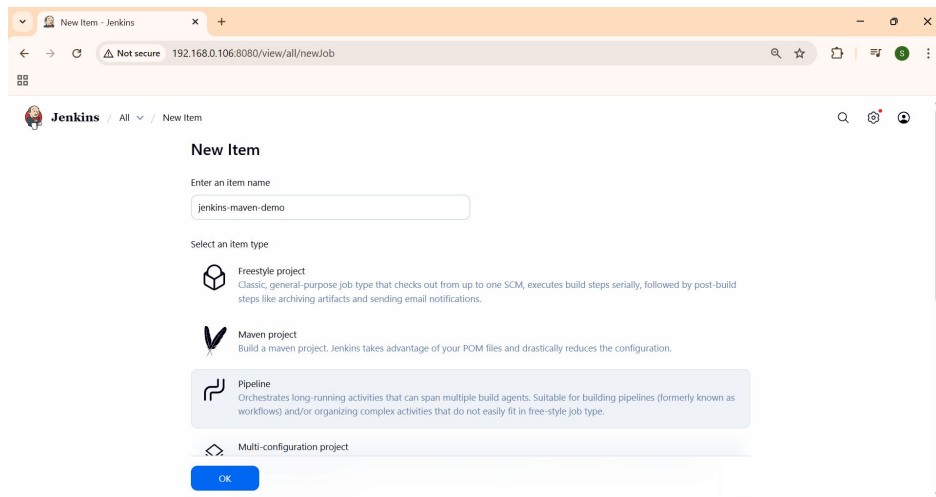
◆ Step 32: Configure Jenkins Job.

1. Open **Jenkins Dashboard**
→ **New Item**



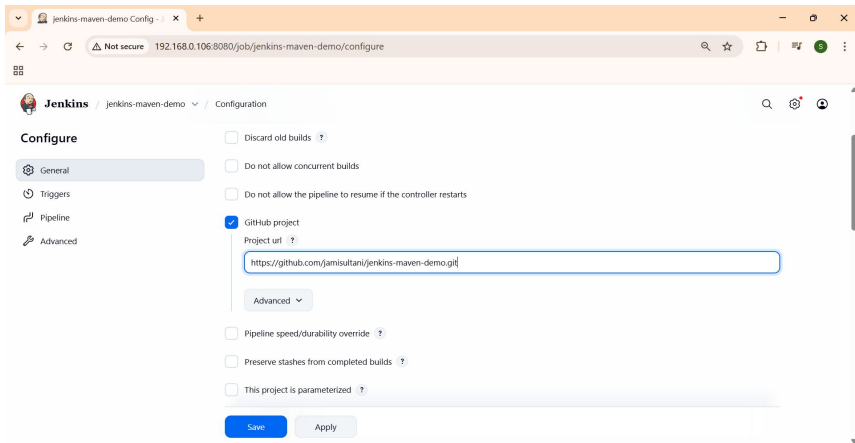
2. Name: **jenkins-maven-demo**

3. Select: **Pipeline** → Click **OK**

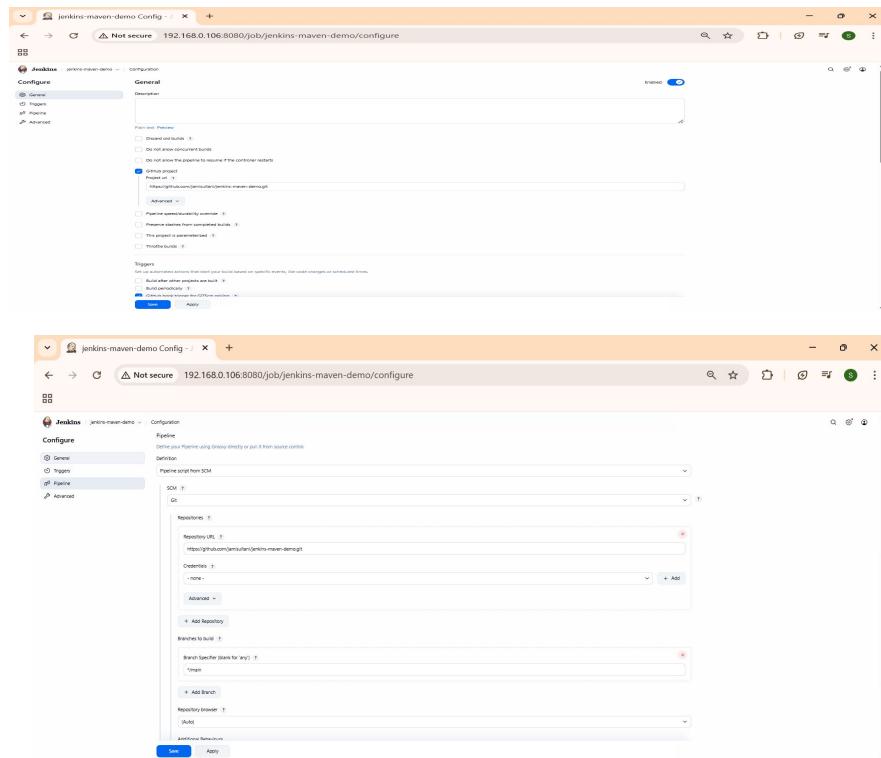


4. Source Code Management → **Git.**

5. Paste your GitHub repo URL.



5. Script Path → **Jenkinsfile**



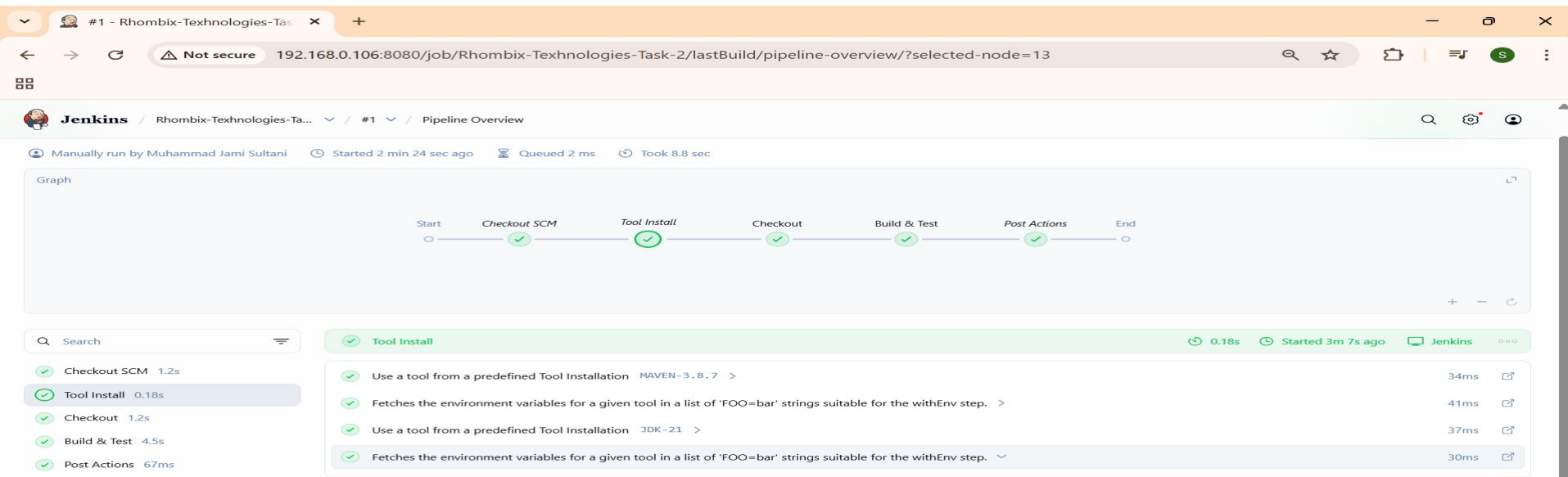
6. Save → **Build Now**



Project Conclusion



I am thrilled to share that my **Jenkins CI pipeline** for the Java Maven project is now fully functional!  This project involved setting up a complete **Continuous Integration (CI) workflow**, where the code is automatically pulled from **GitHub**, built using **Maven**, and tested with **JUnit**.



The screenshot displays the Jenkins Pipeline Overview page for a job named "Rhombix-Technologies-Task-2". The pipeline is shown as a sequence of steps: Start, Checkout SCM, Tool Install, Checkout, Build & Test, Post Actions, and End. All steps are marked with green checkmarks, indicating successful completion. The "Tool Install" step is highlighted in the left sidebar and expanded in the main view, showing two sub-steps: "Use a tool from a predefined Tool Installation MAVEN - 3.8.7" (0.18s) and "Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step." (0.03s). The Jenkins logo and navigation links are visible at the top.

Manually run by Muhammad Jami Sultani | Started 2 min 24 sec ago | Queued 2 ms | Took 8.8 sec

Graph

Start — Checkout SCM — Tool Install — Checkout — Build & Test — Post Actions — End

Search

- Checkout SCM 1.2s
- Tool Install 0.18s**
- Checkout 1.2s
- Build & Test 4.5s
- Post Actions 67ms

Tool Install 0.18s | Started 3m 7s ago | Jenkins

- Use a tool from a predefined Tool Installation MAVEN - 3.8.7 > 34ms
- Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. > 41ms
- Use a tool from a predefined Tool Installation JDK - 21 > 37ms
- Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. > 30ms

Throughout the project, I successfully configured:

1. **JDK 21** 🟢
2. **Maven 3.8.7** 🛠️
3. **Jenkins Pipeline** with a **Jenkinsfile** 📄
4. Automated **build and test stages** 🚀
5. Post-build **success and failure notifications** 🔔

This hands-on project gave me practical experience in automating build processes and integrating tools to create a fully operational CI pipeline, enhancing both efficiency and reliability in software development. 💻 ✨

“End-to-End Jenkins Pipeline: Automated Build & Test for Java Maven Project”



Rhombix-Technologies-Task-2

N/A

N/A

N/A



Dashboard - Jenkins

Not secure 192.168.0.106:8080

Jenkins

+ New Item

Build History

All +

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	jenkins-maven-demo	2 min 29 sec #1	N/A	8.2 sec
✓	☹	jenkins-maven-demo-1	7 min 42 sec #2	26 min #1	1 min 0 sec
✓	☀	myfirstcopysecondproject	1 day 5 hr #1	N/A	20 ms
✓	☀	myfirstproject	1 day 5 hr #5	1 day 5 hr #4	40 ms
✓	☀	Rhombix-Technologies-Task-2	N/A	N/A	N/A
✗	☀	time-tracker	N/A	8 hr 17 min #1	5.1 sec

Build Queue: No builds in the queue.

Build Executor Status: 0/2

Build Now Done.

REST API Jenkins 2.528.1

Build Executor Status

1/2

Rhombix-Technologies-Task

-2

#1

(part) ✗



Rhombix-Technologies-Task-2

#1



✓ Build Success – CI Pipeline Completed!

```
[Pipeline] }
```

```
[Pipeline] // stage
```

```
[Pipeline] }
```

```
[Pipeline] // withEnv
```

```
[Pipeline] }
```

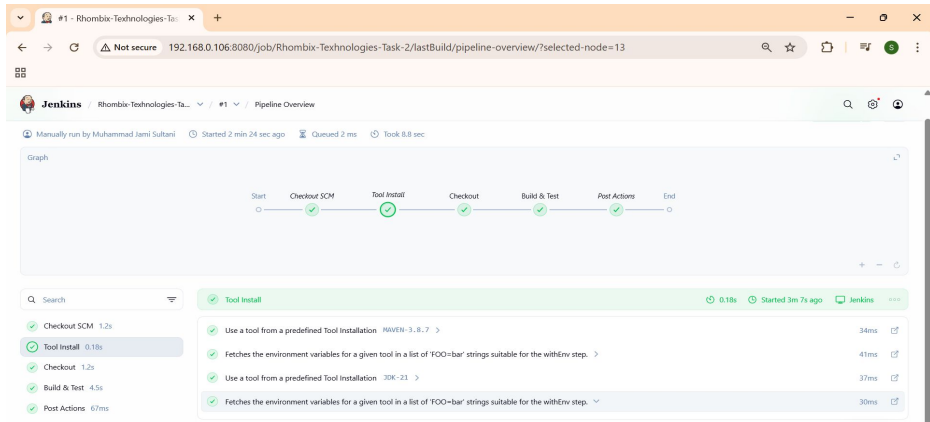
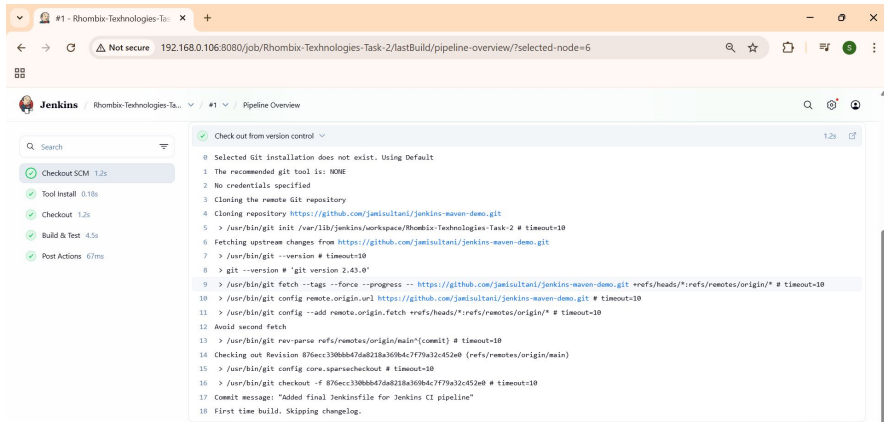
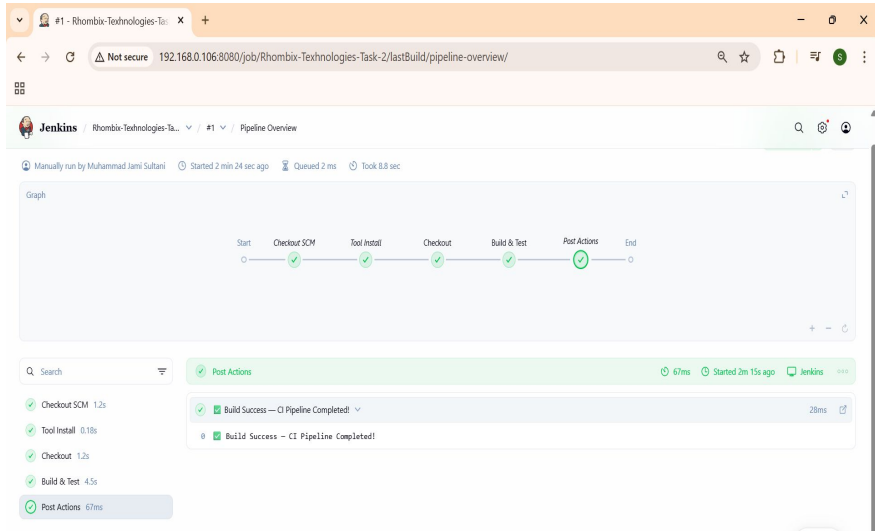
```
[Pipeline] // withEnv
```

```
[Pipeline] }
```

```
[Pipeline] // node
```

```
[Pipeline] End of Pipeline
```

Finished: SUCCESS



Browser window showing Jenkins Pipeline Overview for job Rhombix-Technologies-Task-2, selected node=36. The URL is 192.168.0.106:8080/job/Rhombix-Technologies-Task-2/lastBuild/pipeline-overview/?selected-node=36.

Jenkins Pipeline Overview for job Rhombix-Technologies-Task-2, selected node=36.

Build & Test (4.5s) Started 3m 22s ago Jenkins

Checkout SCM 1.2s

Tool Install 0.18s

Checkout 1.2s

Build & Test 4.5s

Post Actions 67ms

Use a tool from a predefined Tool Installation MAVEN-3.8.7 29ms

Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. 30ms

Use a tool from a predefined Tool Installation JDK-21 26ms

Fetches the environment variables for a given tool in a list of 'FOO=bar' strings suitable for the withEnv step. 33ms

mvn clean install 4.2s

```
0 + mvn clean install
1 [INFO] Scanning for projects...
2 [INFO]
3 [INFO] -----<!t; com.jami.demo:jenkins-maven-demo &gt;-----
4 [INFO] Building jenkins-maven-demo 1.0-SNAPSHOT
5 [INFO] -----[ jar ]-----
6 [INFO]
7 [INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ jenkins-maven-demo ---
8 [INFO] Deleting /var/lib/jenkins/workspace/Rhombix-Technologies-Task-2/target
9 [INFO]
10 [INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ jenkins-maven-demo ---
```

Browser window showing Jenkins Pipeline Overview for job Rhombix-Technologies-Task-2, selected node=36. The URL is 192.168.0.106:8080/job/Rhombix-Technologies-Task-2/lastBuild/pipeline-overview/?selected-node=36.

Jenkins Pipeline Overview for job Rhombix-Technologies-Task-2, selected node=36.

mvn clean install 4.2s

TESTS

Running com.jami.demo.AppTest

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.018 sec

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO]

--- maven-jar-plugin:2.4:jar (default-jar) @ jenkins-maven-demo ---

[INFO] Building jar: /var/lib/jenkins/workspace/Rhombix-Technologies-Task-2/target/jenkins-maven-demo-1.0-SNAPSHOT.jar

[INFO]

[INFO] --- maven-install-plugin:2.4:install (default-install) @ jenkins-maven-demo ---

[INFO]

] Installing /var/lib/jenkins/workspace/Rhombix-Technologies-Task-2/target/jenkins-maven-demo-1.0-SNAPSHOT.jar to /var/lib/jenkins/.m2/repository/com/jami/demo/jenkins-maven-demo-1.0-SNAPSHOT/jenkins-maven-demo-1.0-SNAPSHOT.jar

[INFO]

] Installing /var/lib/jenkins/workspace/Rhombix-Technologies-Task-2/pom.xml to /var/lib/jenkins/.m2/repository/com/jami/demo/jenkins-maven-demo-1.0-SNAPSHOT/jenkins-maven-demo-1.0-SNAPSHOT.pom

[INFO] -----

[INFO] BUILD SUCCESS

[INFO] -----